Ecommerce_Demand_Forecasting_Analysis-Working-Copy

October 23, 2026

[1]:
```
import pandas as pd

# Load the sales dataset
df = pd.read_csv(r"/home/aravind-6928/MBA/Sem 4/Shankar/Data/Rohilik/
 ↪sales_train.csv")

# Display dataset dimensions
print("Dataset shape:", df.shape)

# Display column names
print("\nColumn names:")
print(df.columns.tolist())

# Display first five records
print("\nFirst 5 records:")
display(df.head())
```

```
Dataset shape: (4007419, 14)

Column names:
['unique_id', 'date', 'warehouse', 'total_orders', 'sales', 'sell_price_main',
'availability', 'type_0_discount', 'type_1_discount', 'type_2_discount',
'type_3_discount', 'type_4_discount', 'type_5_discount', 'type_6_discount']

First 5 records:
```

	unique_id	date	warehouse	total_orders	sales	sell_price_main
0	4845	2024-03-10	Budapest_1	6436.0	16.34	646.26
1	4845	2021-05-25	Budapest_1	4663.0	12.63	455.96
2	4845	2021-12-20	Budapest_1	6507.0	34.55	455.96
3	4845	2023-04-29	Budapest_1	5463.0	34.52	646.26
4	4845	2022-04-01	Budapest_1	5997.0	35.92	486.41

	availability	type_0_discount	type_1_discount	type_2_discount
0	1.00	0.00000	0.0	0.0
1	1.00	0.00000	0.0	0.0
2	1.00	0.00000	0.0	0.0
3	0.96	0.20024	0.0	0.0
4	1.00	0.00000	0.0	0.0

	type_3_discount	type_4_discount	type_5_discount	type_6_discount
0	0.0	0.15312	0.0	0.0
1	0.0	0.15025	0.0	0.0
2	0.0	0.15025	0.0	0.0
3	0.0	0.15312	0.0	0.0
4	0.0	0.15649	0.0	0.0

[2]:
```
# Check column data types
print("DATA TYPES")
display(df.dtypes.to_frame(name="Data Type"))

# Check missing values
print("\nMISSING VALUES")
missing = df.isnull().sum().to_frame(name="Missing Count")
missing["Missing %"] = (missing["Missing Count"] / len(df) * 100).round(2)
display(missing)

# Check duplicate records
print("\nDUPLICATE ROWS:", df.duplicated().sum())
```

```
DATA TYPES
```

	Data Type
unique_id	int64
date	str
warehouse	str
total_orders	float64
sales	float64
sell_price_main	float64
availability	float64
type_0_discount	float64
type_1_discount	float64
type_2_discount	float64
type_3_discount	float64
type_4_discount	float64
type_5_discount	float64
type_6_discount	float64

```
MISSING VALUES
```

	Missing Count	Missing %
unique_id	0	0.0
date	0	0.0
warehouse	0	0.0
total_orders	52	0.0
sales	52	0.0
sell_price_main	0	0.0
availability	0	0.0

```
type_0_discount              0        0.0
type_1_discount              0        0.0
type_2_discount              0        0.0
type_3_discount              0        0.0
type_4_discount              0        0.0
type_5_discount              0        0.0
type_6_discount              0        0.0


DUPLICATE ROWS: 0
```

[3]:
```
missing_rows = df[df["sales"].isna() | df["total_orders"].isna()]

print("Missing records:", len(missing_rows))
display(missing_rows.head(10))

print("\nMissing values by column:")
print(df[["sales", "total_orders"]].isna().sum())

print("\nDuplicate rows:", df.duplicated().sum())
```

```
Missing records: 52
```

	unique_id	date	warehouse	total_orders	sales	\
154017	885	2021-05-23	Munich_1	NaN	NaN	
154099	885	2021-05-21	Munich_1	NaN	NaN	
154149	885	2021-05-24	Munich_1	NaN	NaN	
154155	885	2021-05-22	Munich_1	NaN	NaN	
429815	1237	2021-12-10	Frankfurt_1	NaN	NaN	
429860	1237	2021-12-09	Frankfurt_1	NaN	NaN	
733468	725	2021-06-27	Munich_1	NaN	NaN	
734198	725	2021-06-26	Munich_1	NaN	NaN	
738594	3778	2021-05-29	Munich_1	NaN	NaN	
738600	3778	2021-05-23	Munich_1	NaN	NaN	

	sell_price_main	availability	type_0_discount	type_1_discount	\
154017	4.02	1.0	0.0000	0.0	
154099	4.02	1.0	0.0000	0.0	
154149	4.02	1.0	0.0000	0.0	
154155	4.02	1.0	0.0000	0.0	
429815	3.84	0.3	0.0000	0.0	
429860	3.84	1.0	0.0000	0.0	
733468	2.82	1.0	0.1000	0.0	
734198	2.82	1.0	0.1000	0.0	
738594	2.75	1.0	0.2446	0.0	
738600	2.75	1.0	0.0000	0.0	

	type_2_discount	type_3_discount	type_4_discount	type_5_discount	\
154017	0.0	0.0	0.0	0.0	

```
154099              0.0              0.0              0.0              0.0
154149              0.0              0.0              0.0              0.0
154155              0.0              0.0              0.0              0.0
429815              0.0              0.0              0.0              0.0
429860              0.0              0.0              0.0              0.0
733468              0.0              0.0              0.0              0.0
734198              0.0              0.0              0.0              0.0
738594              0.0              0.0              0.0              0.0
738600              0.0              0.0              0.0              0.0

        type_6_discount
154017              0.0
154099              0.0
154149              0.0
154155              0.0
429815              0.0
429860              0.0
733468              0.0
734198              0.0
738594              0.0
738600              0.0


Missing values by column:
sales           52
total_orders    52
dtype: int64

Duplicate rows: 0
```

[4]:
```
df_clean = df.dropna(subset=["sales", "total_orders"]).copy()

print("Original records:", len(df))
print("Records after cleaning:", len(df_clean))
print("Records removed:", len(df) - len(df_clean))
print("Remaining missing values:", df_clean[["sales", "total_orders"]].isna().
  ↪sum().sum())
```

```
Original records: 4007419
Records after cleaning: 4007367
Records removed: 52
Remaining missing values: 0
```

[5]:
```
df_clean["date"] = pd.to_datetime(df_clean["date"])

print("Date data type:", df_clean["date"].dtype)
print("Earliest date:", df_clean["date"].min())
print("Latest date:", df_clean["date"].max())
```

```
print("Unique dates:", df_clean["date"].nunique())
```

```
Date data type: datetime64[us]
Earliest date: 2020-08-01 00:00:00
Latest date: 2024-06-02 00:00:00
Unique dates: 1402
```

[6]:
```
print("Unique products:", df_clean["unique_id"].nunique())
print("Unique warehouses:", df_clean["warehouse"].nunique())

print("\nRecords per warehouse:")
display(df_clean["warehouse"].value_counts().to_frame("Record Count"))

print("\nRecords per product (summary):")
display(df_clean.groupby("unique_id").size().describe())
```

```
Unique products: 5390
Unique warehouses: 7

Records per warehouse:
```

warehouse	Record Count
Prague_1	780566
Prague_3	779655
Prague_2	770709
Brno_1	643637
Budapest_1	574582
Munich_1	259287
Frankfurt_1	198931

```
Records per product (summary):
```

```
count    5390.000000
mean      743.481818
std       496.798923
min         7.000000
25%       260.000000
50%       731.000000
75%      1283.750000
max      1402.000000
dtype: float64
```

0.0.1 Warehouse-level transaction overview

Summarize observed sales and order volume by warehouse. Daily totals aggregate product records; missing warehouse-days are not filled with zero. Each product ID occurs in one warehouse only in this file, so the results support warehouse-level description but not same-product comparison across warehouses. Totals also reflect differing histories and product assortments. The notebook also

reports a record-coverage proxy, where each product is considered active from its first through last observed date; this is a data-completeness diagnostic, not proof of assortment or stock availability.

[7]:
```
import numpy as np

# Confirm mapping and estimate daily record coverage against each product's␣
 ↪observed active span.
product_warehouse_counts = df_clean.groupby("unique_id")["warehouse"].nunique()
print("Products assigned to multiple warehouses:",␣
 ↪int((product_warehouse_counts > 1).sum()))
warehouse_daily = df_clean.groupby(["warehouse", "date"], as_index=False).
 ↪agg(daily_sales=("sales", "sum"), sum_product_orders=("total_orders",␣
 ↪"sum"), product_records=("unique_id", "nunique"))
warehouse_daily["active_product_count_proxy"] = 0
product_activity = (
    df_clean.groupby(["warehouse", "unique_id"])["date"]
    .agg(first_date="min", last_date="max")
    .reset_index()
)
for warehouse, daily_rows in warehouse_daily.groupby("warehouse"):
    activity = product_activity[product_activity["warehouse"] == warehouse]
    first_dates = np.sort(activity["first_date"].
 ↪to_numpy(dtype="datetime64[ns]"))
    last_dates = np.sort(activity["last_date"].to_numpy(dtype="datetime64[ns]"))
    dates = daily_rows["date"].to_numpy(dtype="datetime64[ns]")
    active_counts = (
        np.searchsorted(first_dates, dates, side="right")
        - np.searchsorted(last_dates, dates, side="left")
    )
    warehouse_daily.loc[daily_rows.index, "active_product_count_proxy"] =␣
 ↪active_counts
warehouse_daily["record_coverage_proxy"] = (
    warehouse_daily["product_records"]
    / warehouse_daily["active_product_count_proxy"].replace(0, np.nan)
)
warehouse_summary = warehouse_daily.groupby("warehouse").
 ↪agg(observed_days=("date", "nunique"), first_date=("date", "min"),␣
 ↪last_date=("date", "max"), total_sales=("daily_sales", "sum"),␣
 ↪mean_sales_per_observed_day=("daily_sales", "mean"),␣
 ↪summed_product_orders=("sum_product_orders", "sum"),␣
 ↪mean_product_orders_per_observed_day=("sum_product_orders", "mean"),␣
 ↪product_records=("product_records", "sum"),␣
 ↪mean_record_coverage_proxy=("record_coverage_proxy", "mean"),␣
 ↪days_at_least_99pct_coverage=("record_coverage_proxy", lambda s: int((s >= 0.
 ↪99).sum())))
warehouse_summary["calendar_days_in_span"] = (warehouse_summary["last_date"] -␣
 ↪warehouse_summary["first_date"]).dt.days + 1
```

```
warehouse_summary["missing_warehouse_dates"] =␣
 ↪warehouse_summary["calendar_days_in_span"] -␣
 ↪warehouse_summary["observed_days"]
warehouse_summary["unique_products"] = df_clean.
 ↪groupby("warehouse")["unique_id"].nunique()
display(warehouse_summary.sort_values("total_sales", ascending=False).round(2))
```

```
Products assigned to multiple warehouses: 0
```

```
/tmp/ipykernel_607394/1856724241.py:31: UserWarning: obj.round has no effect
with datetime, timedelta, or period dtypes. Use obj.dt.round(…) instead.
  display(warehouse_summary.sort_values("total_sales",
ascending=False).round(2))
```

warehouse	observed_days	first_date	last_date	total_sales	mean_sales_per_observed_day	summed_product_orders	mean_product_orders_per_observed_day	product_records	mean_record_coverage_proxy	days_at_least_99pct_coverage
Prague_1	1402	2020-08-01	2024-06-02	1.144583e+08	81639.33	7.114396e+09	5074462.29	780566	0.84	3
Brno_1	1402	2020-08-01	2024-06-02	1.067244e+08	76122.96	5.275866e+09	3763099.85	643637	0.83	2
Prague_3	1402	2020-08-01	2024-06-02	6.105859e+07	43551.06	3.791152e+09	2704103.03	779655		
Budapest_1	1402	2020-08-01	2024-06-02	5.914415e+07	42185.56	3.170344e+09	2261301.20	574582		
Prague_2	1402	2020-08-01	2024-06-02	5.795784e+07	41339.40	3.567743e+09	2544752.41	770709		
Munich_1	1087	2021-05-20	2024-06-02	2.580422e+07	23738.93	8.248630e+08	758843.64	259287		
Frankfurt_1	853	2021-12-08	2024-06-02	9.173921e+06	10754.89	2.826942e+08	331411.68	198931		

```
Prague_3                        0.84                          2
Budapest_1                      0.73                          2
Prague_2                        0.84                          2
Munich_1                        0.72                          3
Frankfurt_1                     0.80                          2
```

warehouse	calendar_days_in_span	missing_warehouse_dates	unique_products
Prague_1	1402	0	857
Brno_1	1402	0	757
Prague_3	1402	0	860
Budapest_1	1402	0	940
Prague_2	1402	0	853
Munich_1	1110	23	640
Frankfurt_1	908	55	483

[8]:
```python
product_id = 4845

product_df = df_clean[df_clean["unique_id"] == product_id].copy()
product_df = product_df.sort_values("date")

print("Product ID:", product_id)
print("Records:", len(product_df))
print("Warehouses:", product_df["warehouse"].unique())
print("First date:", product_df["date"].min())
print("Last date:", product_df["date"].max())
print("Zero-sales records:", (product_df["sales"] == 0).sum())
print("Average sales:", round(product_df["sales"].mean(), 2))
print("Median sales:", round(product_df["sales"].median(), 2))

display(product_df[["date", "warehouse", "sales"]].head(10))
```

```
Product ID: 4845
Records: 1374
Warehouses: <StringArray>
['Budapest_1']
Length: 1, dtype: str
First date: 2020-08-01 00:00:00
Last date: 2024-06-02 00:00:00
Zero-sales records: 11
Average sales: 21.75
Median sales: 20.66
```

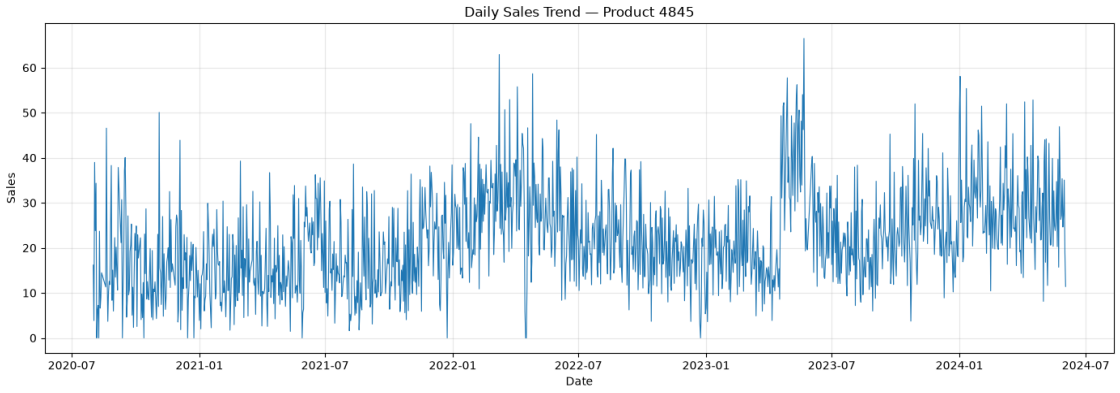
	date	warehouse	sales
897	2020-08-01	Budapest_1	16.17
176	2020-08-02	Budapest_1	3.85
599	2020-08-03	Budapest_1	38.97
276	2020-08-04	Budapest_1	23.81
584	2020-08-05	Budapest_1	34.34

```
715 2020-08-06  Budapest_1   0.00
903 2020-08-07  Budapest_1   6.17
618 2020-08-08  Budapest_1   7.23
385 2020-08-09  Budapest_1   0.00
926 2020-08-10  Budapest_1  23.69
```

[9]:
```python
import matplotlib.pyplot as plt

plt.figure(figsize=(14, 5))
plt.plot(product_df["date"], product_df["sales"], linewidth=0.8)

plt.title("Daily Sales Trend - Product 4845")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



[10]:
```python
all_dates = pd.date_range(
    start=product_df["date"].min(),
    end=product_df["date"].max(),
    freq="D"
)

existing_dates = pd.DatetimeIndex(product_df["date"].unique())

missing_dates = all_dates.difference(existing_dates)

print("Expected calendar days:", len(all_dates))
print("Existing dates:", len(existing_dates))
print("Missing dates:", len(missing_dates))
print("\nFirst 20 missing dates:")
print(missing_dates[:20].tolist())
```

```
Expected calendar days: 1402
Existing dates: 1374
Missing dates: 28

First 20 missing dates:
[Timestamp('2020-08-14 00:00:00'), Timestamp('2020-08-15 00:00:00'),
Timestamp('2020-08-16 00:00:00'), Timestamp('2020-08-17 00:00:00'),
Timestamp('2020-08-18 00:00:00'), Timestamp('2020-08-26 00:00:00'),
Timestamp('2020-09-07 00:00:00'), Timestamp('2020-09-08 00:00:00'),
Timestamp('2020-09-09 00:00:00'), Timestamp('2020-10-06 00:00:00'),
Timestamp('2020-10-07 00:00:00'), Timestamp('2020-11-24 00:00:00'),
Timestamp('2020-11-25 00:00:00'), Timestamp('2020-12-28 00:00:00'),
Timestamp('2021-01-17 00:00:00'), Timestamp('2021-03-16 00:00:00'),
Timestamp('2021-03-17 00:00:00'), Timestamp('2021-06-23 00:00:00'),
Timestamp('2021-12-28 00:00:00'), Timestamp('2023-04-25 00:00:00')]
```

[11]:
```
missing_dates_df = pd.DataFrame({"date": missing_dates})

missing_dates_df["gap_group"] = (
    missing_dates_df["date"].diff().dt.days.ne(1)
).cumsum()

gap_summary = (
    missing_dates_df.groupby("gap_group")
    .agg(
        start_date=("date", "min"),
        end_date=("date", "max"),
        missing_days=("date", "count")
    )
    .reset_index(drop=True)
)

display(gap_summary)
```

	start_date	end_date	missing_days
0	2020-08-14	2020-08-18	5
1	2020-08-26	2020-08-26	1
2	2020-09-07	2020-09-09	3
3	2020-10-06	2020-10-07	2
4	2020-11-24	2020-11-25	2
5	2020-12-28	2020-12-28	1
6	2021-01-17	2021-01-17	1
7	2021-03-16	2021-03-17	2
8	2021-06-23	2021-06-23	1
9	2021-12-28	2021-12-28	1
10	2023-04-25	2023-04-27	3
11	2023-05-28	2023-06-01	5
12	2024-04-11	2024-04-11	1

[12]:
```
print("SALES DISTRIBUTION")

display(product_df["sales"].describe().to_frame().round(2))

print("\nZero-sales records:", (product_df["sales"] == 0).sum())
print("Negative-sales records:", (product_df["sales"] < 0).sum())

print("\nSales percentiles:")
display(
    product_df["sales"]
    .quantile([0.01, 0.05, 0.25, 0.50, 0.75, 0.95, 0.99])
    .to_frame(name="Sales")
    .round(2)
)
```

```
SALES DISTRIBUTION
```

	sales
count	1374.00
mean	21.75
std	10.68
min	0.00
25%	13.65
50%	20.66
75%	28.38
max	66.49

```
Zero-sales records: 11
Negative-sales records: 0

Sales percentiles:
```

	Sales
0.01	1.77
0.05	6.04
0.25	13.65
0.50	20.66
0.75	28.38
0.95	39.93
0.99	52.04

[13]:
```
product_df = product_df.sort_values("date").reset_index(drop=True)

train_df = product_df.iloc[:-90].copy()
test_df = product_df.iloc[-90:].copy()

print("Training records:", len(train_df))
print("Training period:", train_df["date"].min(), "to", train_df["date"].max())
```

```
print("\nTesting records:", len(test_df))
print("Testing period:", test_df["date"].min(), "to", test_df["date"].max())

print("\nDays between first and last test date:",
      (test_df["date"].max() - test_df["date"].min()).days + 1)

print("Missing calendar days in test period:",
      ((test_df["date"].max() - test_df["date"].min()).days + 1) - len(test_df))
```

```
Training records: 1284
Training period: 2020-08-01 00:00:00 to 2024-03-03 00:00:00

Testing records: 90
Testing period: 2024-03-04 00:00:00 to 2024-06-02 00:00:00

Days between first and last test date: 91
Missing calendar days in test period: 1
```

[14]:
```
test_calendar = pd.date_range(
    start=test_df["date"].min(),
    end=test_df["date"].max(),
    freq="D"
)

missing_test_dates = test_calendar.difference(
    pd.DatetimeIndex(test_df["date"])
)

print("Missing dates in test period:")
print(missing_test_dates.tolist())
```

```
Missing dates in test period:
[Timestamp('2024-04-11 00:00:00')]
```

[15]:
```
window = 7

last_7_sales = train_df["sales"].tail(window)
moving_average_forecast = last_7_sales.mean()

print("Last 7 training sales:")
print(last_7_sales.tolist())

print("\n7-day Moving Average forecast:", round(moving_average_forecast, 2))
print("Test records:", len(test_df))
```

```
Last 7 training sales:
[19.67, 26.16, 30.67, 42.38, 21.03, 27.26, 20.59]
```

```
7-day Moving Average forecast: 26.82
Test records: 90
```

[16]:
```python
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

# Same forecast for all test dates
y_true = test_df["sales"].to_numpy()
y_pred = np.full(len(test_df), moving_average_forecast)

mae = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))

# MAPE is undefined when actual sales are zero
nonzero_mask = y_true != 0
mape = np.mean(
    np.abs((y_true[nonzero_mask] - y_pred[nonzero_mask]) / y_true[nonzero_mask])
) * 100

print("Model: 7-day Moving Average Baseline")
print("MAE:", round(mae, 2))
print("RMSE:", round(rmse, 2))
print("MAPE (%):", round(mape, 2))
print("Test observations:", len(y_true))
print("Zero-sales observations excluded from MAPE:", int((~nonzero_mask).sum()))
```

```
Model: 7-day Moving Average Baseline
MAE: 7.59
RMSE: 9.8
MAPE (%): 29.65
Test observations: 90
Zero-sales observations excluded from MAPE: 0
```

[17]:
```python
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

model = ExponentialSmoothing(
    train_df["sales"],
    trend=None,
    seasonal=None,
    initialization_method="estimated"
)

fitted_model = model.fit()

es_forecast = fitted_model.forecast(len(test_df))
```

```
y_true = test_df["sales"].to_numpy()
y_pred_es = np.asarray(es_forecast)

mae_es = mean_absolute_error(y_true, y_pred_es)
rmse_es = np.sqrt(mean_squared_error(y_true, y_pred_es))

nonzero_mask = y_true != 0
mape_es = np.mean(
    np.abs(
        (y_true[nonzero_mask] - y_pred_es[nonzero_mask])
        / y_true[nonzero_mask]
    )
) * 100

print("Model: Simple Exponential Smoothing")
print("MAE:", round(mae_es, 2))
print("RMSE:", round(rmse_es, 2))
print("MAPE (%):", round(mape_es, 2))
print("Test observations:", len(y_true))
```

```
Model: Simple Exponential Smoothing
MAE: 7.68
RMSE: 9.92
MAPE (%): 29.43
Test observations: 90
```

[18]:
```
import matplotlib.pyplot as plt

plt.figure(figsize=(14, 6))

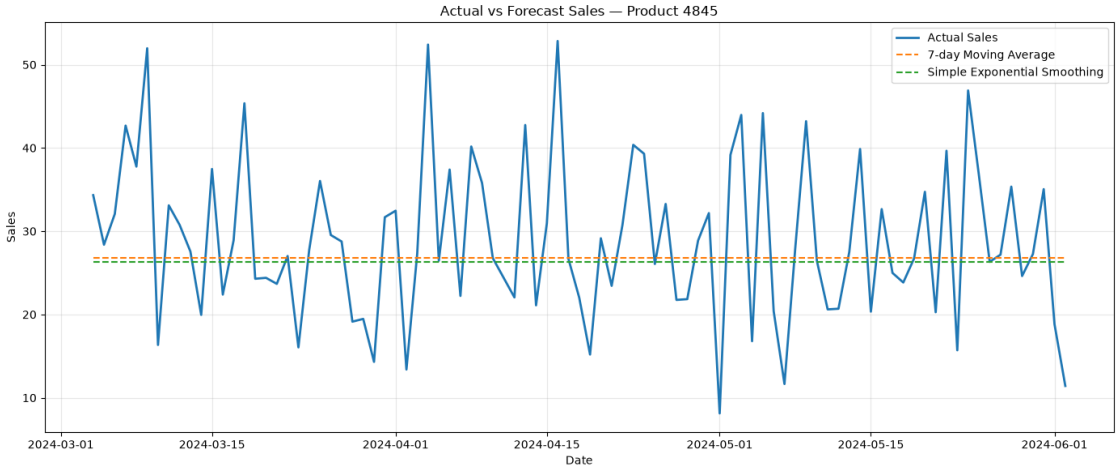
plt.plot(
    test_df["date"],
    test_df["sales"],
    label="Actual Sales",
    linewidth=2
)

plt.plot(
    test_df["date"],
    np.full(len(test_df), moving_average_forecast),
    label="7-day Moving Average",
    linestyle="--"
)

plt.plot(
    test_df["date"],
    y_pred_es,
```

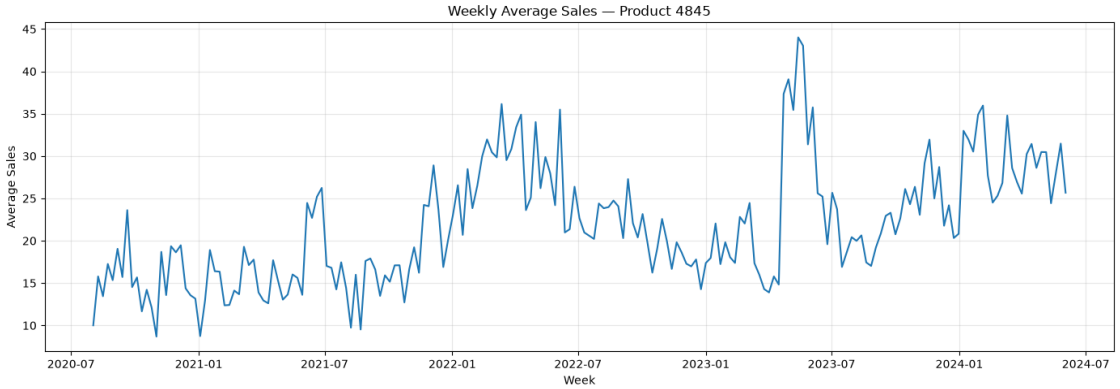
```
    label="Simple Exponential Smoothing",
    linestyle="--"
)

plt.title("Actual vs Forecast Sales - Product 4845")
plt.xlabel("Date")
plt.ylabel("Sales")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



[19]:
```
weekly_sales = (
    product_df
    .set_index("date")["sales"]
    .resample("W")
    .mean()
)

plt.figure(figsize=(14, 5))
plt.plot(weekly_sales.index, weekly_sales.values)
plt.title("Weekly Average Sales - Product 4845")
plt.xlabel("Week")
plt.ylabel("Average Sales")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



[20]:
```
product_df["day_name"] = product_df["date"].dt.day_name()

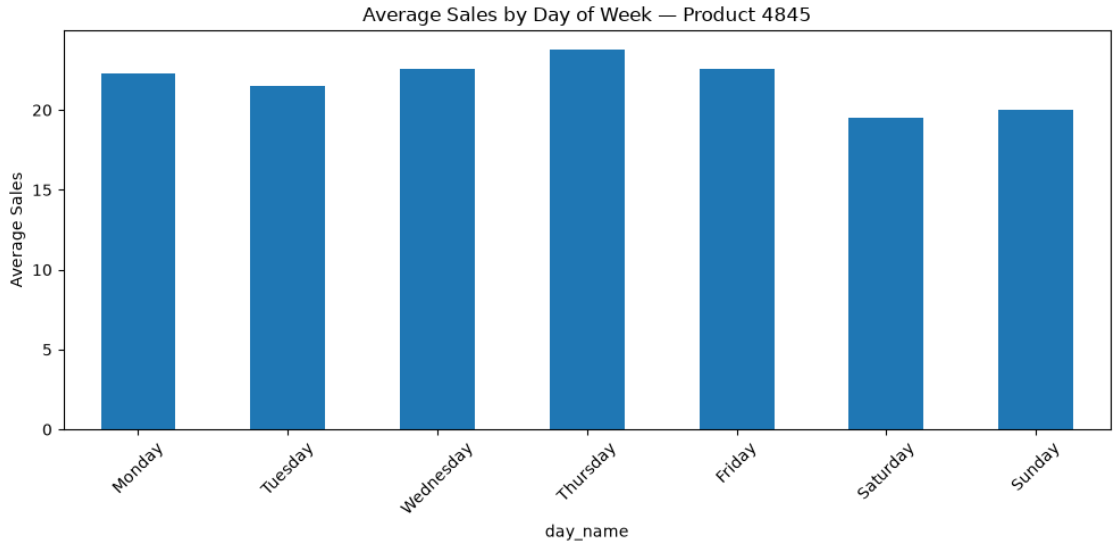
weekday_sales = (
    product_df.groupby("day_name")["sales"]
    .agg(["mean", "median", "count"])
    .reindex([
        "Monday", "Tuesday", "Wednesday",
        "Thursday", "Friday", "Saturday", "Sunday"
    ])
)

display(weekday_sales.round(2))

weekday_sales["mean"].plot(
    kind="bar",
    figsize=(10, 5),
    title="Average Sales by Day of Week - Product 4845",
    ylabel="Average Sales"
)

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

day_name	mean	median	count
Monday	22.31	21.58	196
Tuesday	21.52	20.30	192
Wednesday	22.57	20.61	192
Thursday	23.78	22.56	197
Friday	22.55	21.54	199
Saturday	19.53	19.63	200
Sunday	20.03	19.08	198



[21]:
```
from scipy.stats import kruskal

weekday_groups = [
    group["sales"].values
    for _, group in product_df.groupby(
        product_df["date"].dt.dayofweek
    )
]

statistic, p_value = kruskal(*weekday_groups)

print("Kruskal-Wallis test")
print("Test statistic:", round(statistic, 3))
print("P-value:", round(p_value, 4))

if p_value < 0.05:
    print("Result: Sales distributions differ significantly across weekdays.")
else:
    print("Result: No statistically significant weekday difference detected.")
```

```
Kruskal-Wallis test
Test statistic: 24.488
P-value: 0.0004
Result: Sales distributions differ significantly across weekdays.
```

0.1 4. Weekday Sales Pattern Analysis

Average sales varied across the days of the week for product 4845. Thursday recorded the highest average sales (23.78), while Saturday recorded the lowest (19.53).

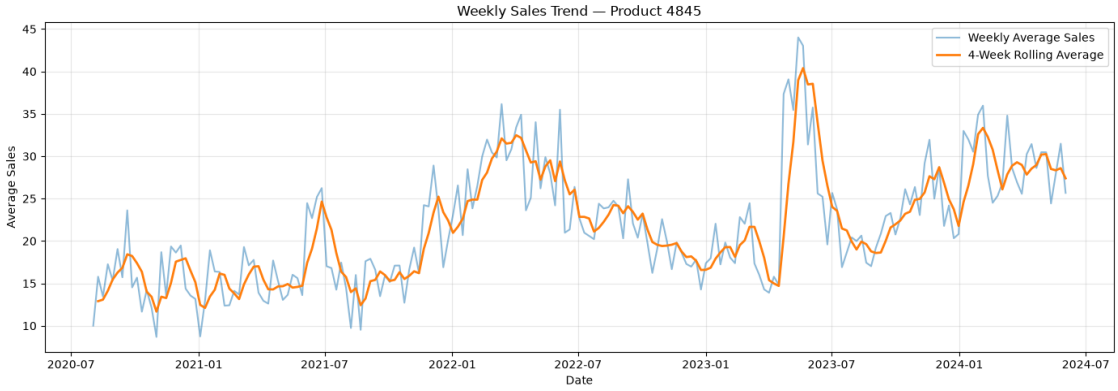
The Kruskal–Wallis test indicated a statistically significant difference in sales distributions across weekdays ($H = 24.488$, $p = 0.0004$). This suggests that weekday related variation may be relevant to demand forecasting. Further validation is required before treating this pattern as stable weekly seasonality.

[22]:
```
weekly_sales = (
    product_df
    .set_index("date")["sales"]
    .resample("W")
    .mean()
)

weekly_rolling_avg = weekly_sales.rolling(
    window=4,
    min_periods=2
).mean()

plt.figure(figsize=(14, 5))
plt.plot(
    weekly_sales.index,
    weekly_sales.values,
    alpha=0.5,
    label="Weekly Average Sales"
)
plt.plot(
    weekly_rolling_avg.index,
    weekly_rolling_avg.values,
    linewidth=2,
    label="4-Week Rolling Average"
)

plt.title("Weekly Sales Trend - Product 4845")
plt.xlabel("Date")
plt.ylabel("Average Sales")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```



[23]:
```python
from scipy.stats import spearmanr

valid_weekly = weekly_sales.dropna()

correlation, p_value = spearmanr(
    np.arange(len(valid_weekly)),
    valid_weekly.values
)

print("Weekly Sales Trend Test")
print("Spearman correlation:", round(correlation, 3))
print("P-value:", round(p_value, 4))

if p_value < 0.05:
    if correlation > 0:
        print("Result: Significant upward monotonic trend detected.")
    else:
        print("Result: Significant downward monotonic trend detected.")
else:
    print("Result: No significant monotonic trend detected.")
```

```
Weekly Sales Trend Test
Spearman correlation: 0.603
P-value: 0.0
Result: Significant upward monotonic trend detected.
```

[24]:
```python
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

holt_model = ExponentialSmoothing(
    train_df["sales"],
    trend="add",
```

```
    seasonal=None,
    initialization_method="estimated"
)

holt_fit = holt_model.fit()

holt_forecast = holt_fit.forecast(len(test_df)).to_numpy()

y_true = test_df["sales"].to_numpy()

mae_holt = mean_absolute_error(y_true, holt_forecast)
rmse_holt = np.sqrt(mean_squared_error(y_true, holt_forecast))

nonzero_mask = y_true != 0
mape_holt = np.mean(
    np.abs(
        (y_true[nonzero_mask] - holt_forecast[nonzero_mask])
        / y_true[nonzero_mask]
    )
) * 100

print("Model: Holt's Linear Trend")
print("MAE:", round(mae_holt, 2))
print("RMSE:", round(rmse_holt, 2))
print("MAPE (%):", round(mape_holt, 2))
print("Test observations:", len(y_true))
```

```
Model: Holt's Linear Trend
MAE: 7.63
RMSE: 9.84
MAPE (%): 29.76
Test observations: 90
```

[25]:
```
# Build a calendar-day series so a 7-day lag means
# the same weekday in the previous week.
daily_sales = (
    product_df.set_index("date")["sales"]
    .sort_index()
    .asfreq("D")
)

# Keep missing dates as missing; do not treat them as zero.
daily_train = daily_sales.loc[:train_df["date"].max()]
daily_test = daily_sales.loc[test_df["date"].min():test_df["date"].max()]

# Seasonal-naive forecast: value from 7 calendar days earlier.
seasonal_naive_forecast = daily_sales.shift(7).reindex(daily_test.index)
```

```
# Evaluate only dates with both actual sales and a valid lagged forecast.
eval_df = pd.DataFrame({
    "actual": daily_test,
    "forecast": seasonal_naive_forecast
}).dropna()

mae_sn = mean_absolute_error(eval_df["actual"], eval_df["forecast"])
rmse_sn = np.sqrt(mean_squared_error(eval_df["actual"], eval_df["forecast"]))

nonzero_mask = eval_df["actual"] != 0
mape_sn = np.mean(
    np.abs(
        (eval_df.loc[nonzero_mask, "actual"] -
         eval_df.loc[nonzero_mask, "forecast"]) /
        eval_df.loc[nonzero_mask, "actual"]
    )
) * 100

print("Model: Seasonal Naive (7-day lag)")
print("MAE:", round(mae_sn, 2))
print("RMSE:", round(rmse_sn, 2))
print("MAPE (%):", round(mape_sn, 2))
print("Evaluated observations:", len(eval_df))
print("Test dates excluded due to missing actual/lagged values:",
      len(daily_test) - len(eval_df))
```

```
Model: Seasonal Naive (7-day lag)
MAE: 10.17
RMSE: 13.01
MAPE (%): 40.94
Evaluated observations: 89
Test dates excluded due to missing actual/lagged values: 2
```

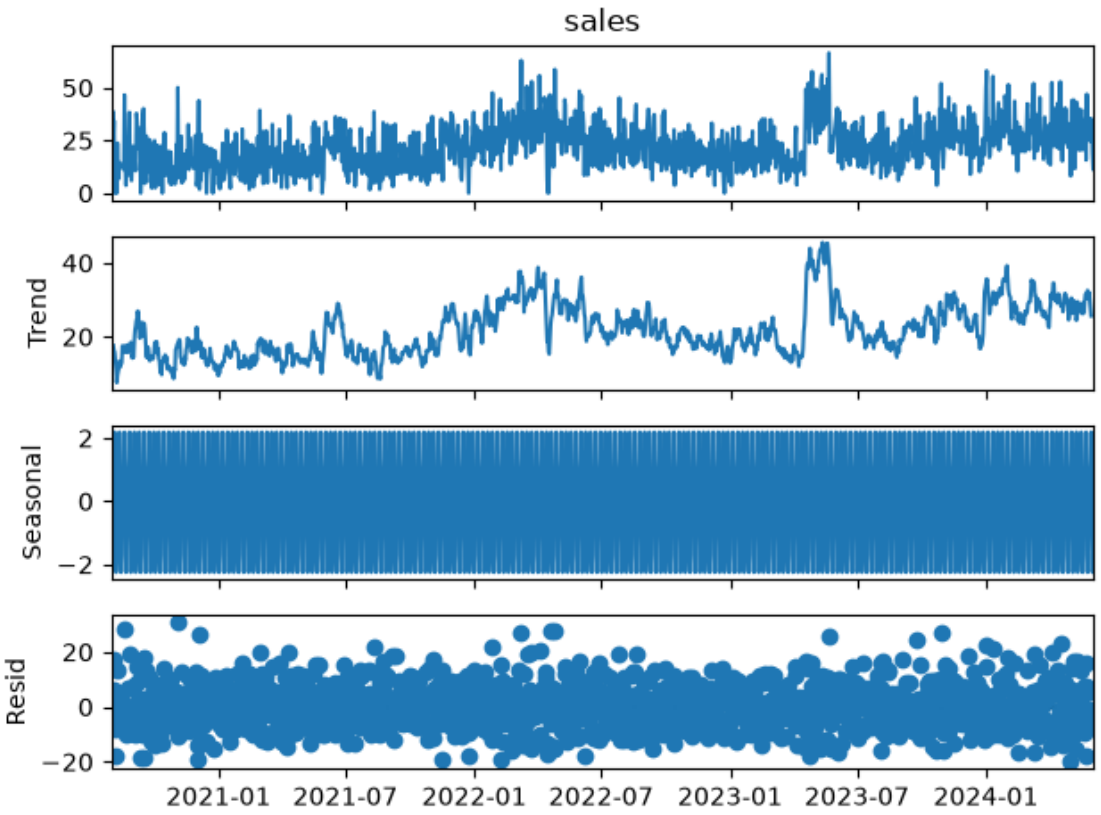
[26]:
```
from statsmodels.tsa.seasonal import seasonal_decompose

# Use a continuous daily series, preserving missing values.
daily_sales = (
    product_df.set_index("date")["sales"]
    .sort_index()
    .asfreq("D")
)

# Fill missing dates only for this exploratory decomposition.
# Interpolation is for visualization/diagnostics, not final model evaluation.
daily_interpolated = daily_sales.interpolate(method="time")
```

```
decomposition = seasonal_decompose(
    daily_interpolated,
    model="additive",
    period=7
)

decomposition.plot()
plt.tight_layout()
plt.show()
```



[27]:
```
seasonal_component = decomposition.seasonal
residual_component = decomposition.resid

valid = pd.concat(
    [seasonal_component, residual_component],
    axis=1
).dropna()

seasonal_strength = max(
    0,
```

```
    1 - valid.iloc[:, 1].var() /
    (valid.iloc[:, 0].var() + valid.iloc[:, 1].var())
)

print("Estimated weekly seasonal strength:",
      round(seasonal_strength, 3))
```

```
Estimated weekly seasonal strength: 0.032
```

[28]:
```
from statsmodels.tsa.holtwinters import ExponentialSmoothing
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

hw_model = ExponentialSmoothing(
    train_df["sales"],
    trend="add",
    seasonal="add",
    seasonal_periods=7,
    initialization_method="estimated"
)

hw_fit = hw_model.fit()

hw_forecast = hw_fit.forecast(len(test_df)).to_numpy()

y_true = test_df["sales"].to_numpy()

mae_hw = mean_absolute_error(y_true, hw_forecast)
rmse_hw = np.sqrt(mean_squared_error(y_true, hw_forecast))

nonzero_mask = y_true != 0
mape_hw = np.mean(
    np.abs(
        (y_true[nonzero_mask] - hw_forecast[nonzero_mask])
        / y_true[nonzero_mask]
    )
) * 100

print("Model: Holt-Winters Additive")
print("MAE:", round(mae_hw, 2))
print("RMSE:", round(rmse_hw, 2))
print("MAPE (%):", round(mape_hw, 2))
print("Test observations:", len(y_true))
```

```
Model: Holt-Winters Additive
MAE: 7.85
RMSE: 10.0
MAPE (%): 30.28
```

```
Test observations: 90
```

[29]:
```python
# Error = actual - forecast
ma_errors = y_true - np.full(len(y_true), moving_average_forecast)
ses_errors = y_true - np.asarray(es_forecast)
holt_errors = y_true - holt_forecast
hw_errors = y_true - hw_forecast

error_summary = pd.DataFrame({
    "Model": [
        "Moving Average",
        "Simple Exponential Smoothing",
        "Holt Linear Trend",
        "Holt-Winters Additive"
    ],
    "Mean Error": [
        ma_errors.mean(),
        ses_errors.mean(),
        holt_errors.mean(),
        hw_errors.mean()
    ],
    "Mean Absolute Error": [
        np.abs(ma_errors).mean(),
        np.abs(ses_errors).mean(),
        np.abs(holt_errors).mean(),
        np.abs(hw_errors).mean()
    ]
})

display(error_summary.round(2))
```

	Model	Mean Error	Mean Absolute Error
0	Moving Average	2.12	7.59
1	Simple Exponential Smoothing	2.62	7.68
2	Holt Linear Trend	2.18	7.63
3	Holt-Winters Additive	2.30	7.85

[30]:
```python
product_summary = (
    df_clean.groupby("unique_id")["sales"]
    .agg(
        observations="count",
        mean_sales="mean",
        median_sales="median",
        std_sales="std"
    )
)

product_summary["coefficient_of_variation"] = (
```

```
    product_summary["std_sales"] / product_summary["mean_sales"]
)

display(
    product_summary[
        product_summary["observations"] >= 365
    ].describe().round(2)
)
```

```
       observations  mean_sales  median_sales  std_sales  \
count       3728.00     3728.00       3728.00    3728.00
mean        1010.08      100.41         89.96      54.59
std          348.71      302.67        292.37     108.69
min          365.00        3.33          0.00       2.67
25%          684.00       22.17         19.28      13.43
50%         1076.00       42.41         36.98      26.04
75%         1358.00       93.19         80.42      57.07
max         1402.00    12283.37      11996.26    2916.89

       coefficient_of_variation
count                   3728.00
mean                       0.67
std                        0.36
min                        0.19
25%                        0.45
50%                        0.57
75%                        0.77
max                        4.77
```

Product selection note The initial stratified-sampling attempt was superseded by the corrected selection cell immediately below. Use that following cell; this note prevents the earlier KeyError from recurring in a clean run.

[31]:
```
eligible_products = product_summary[
    (product_summary["observations"] >= 365) &
    (product_summary["mean_sales"] > 0) &
    (product_summary["coefficient_of_variation"].notna())
].copy()

eligible_products["variability_group"] = pd.qcut(
    eligible_products["coefficient_of_variation"],
    q=3,
    labels=["Low variability", "Medium variability", "High variability"]
)

selected_products = pd.concat([
    group.sample(n=5, random_state=42)
```

```
    for _, group in eligible_products.groupby("variability_group",␣
  ↪observed=True)
])

selected_products = selected_products.reset_index()

display(
    selected_products[
        [
            "unique_id",
            "observations",
            "mean_sales",
            "coefficient_of_variation",
            "variability_group"
        ]
    ].round(2)
)
```

	unique_id	observations	mean_sales	coefficient_of_variation	\
0	4067	708	48.87	0.44	
1	4425	997	294.03	0.27	
2	2990	1352	20.05	0.46	
3	2792	998	36.91	0.40	
4	4136	996	34.42	0.45	
5	3846	647	13.18	0.62	
6	4345	1003	66.82	0.53	
7	3000	1170	25.32	0.53	
8	2723	1372	16.62	0.55	
9	4002	629	70.33	0.57	
10	3788	457	48.05	0.74	
11	4361	742	22.08	0.82	
12	2782	653	10.45	1.97	
13	2627	1223	47.29	0.69	
14	3956	850	205.67	0.77	

	variability_group
0	Low variability
1	Low variability
2	Low variability
3	Low variability
4	Low variability
5	Medium variability
6	Medium variability
7	Medium variability
8	Medium variability
9	Medium variability
10	High variability
11	High variability

```
12    High variability
13    High variability
14    High variability
```

[32]:
```
print("product_summary columns:")
print(product_summary.columns.tolist())

print("\nproduct_summary index name:")
print(product_summary.index.name)

print("\nFirst 5 rows:")
display(product_summary.head())

print("\nSelected products columns:")
print(selected_products.columns.tolist())
```

```
product_summary columns:
['observations', 'mean_sales', 'median_sales', 'std_sales',
'coefficient_of_variation']

product_summary index name:
unique_id

First 5 rows:
```

unique_id	observations	mean_sales	median_sales	std_sales	coefficient_of_variation
0	35	213.986571	127.35	179.877570	0.840602
1	189	93.396772	91.10	58.748743	0.629023
2	51	209.623922	188.95	65.498724	0.312458
3	373	92.173137	94.58	51.483404	0.558551
5	483	48.754265	48.46	16.258250	0.333473

```
Selected products columns:
['unique_id', 'observations', 'mean_sales', 'median_sales', 'std_sales',
'coefficient_of_variation', 'variability_group']
```

[33]:
```
selected_product_ids = selected_products["unique_id"].tolist()

print("Number of selected products:", len(selected_product_ids))
```

```
print("Selected product IDs:", selected_product_ids)
```

```
Number of selected products: 15
Selected product IDs: [4067, 4425, 2990, 2792, 4136, 3846, 4345, 3000, 2723,
4002, 3788, 4361, 2782, 2627, 3956]
```

[34]:
```
selected_product_summary = (
    df_clean[df_clean["unique_id"].isin(selected_product_ids)]
    .groupby("unique_id")
    .agg(
        observations=("date", "nunique"),
        first_date=("date", "min"),
        last_date=("date", "max"),
        mean_sales=("sales", "mean")
    )
)

selected_product_summary["calendar_days"] = (
    selected_product_summary["last_date"]
    - selected_product_summary["first_date"]
).dt.days + 1

selected_product_summary["missing_dates"] = (
    selected_product_summary["calendar_days"]
    - selected_product_summary["observations"]
)

display(selected_product_summary)
```

```
           observations first_date  last_date  mean_sales  calendar_days  \
unique_id
2627               1223 2020-08-01 2024-06-02   47.292469           1402
2723               1372 2020-08-01 2024-06-02   16.615255           1402
2782                653 2020-08-01 2024-06-02   10.452940           1402
2792                998 2021-09-06 2024-06-02   36.910461           1001
2990               1352 2020-08-01 2024-05-30   20.051102           1399
3000               1170 2020-08-01 2024-06-02   25.323479           1402
3788                457 2021-11-25 2023-04-03   48.053173            495
3846                647 2022-03-14 2024-05-13   13.177929            792
3956                850 2020-08-01 2024-05-29  205.669224           1398
4002                629 2022-08-24 2024-06-02   70.325978            649
4067                708 2022-06-24 2024-06-02   48.874393            710
4136                996 2020-08-01 2024-06-02   34.416145           1402
4345               1003 2021-08-02 2024-06-02   66.819621           1036
4361                742 2020-11-06 2023-06-25   22.081186            962
4425                997 2021-09-07 2024-06-02  294.031836           1000

           missing_dates
```

```
unique_id
2627                179
2723                 30
2782                749
2792                  3
2990                 47
3000                232
3788                 38
3846                145
3956                548
4002                 20
4067                  2
4136                406
4345                 33
4361                220
4425                  3
```

Date coverage was examined for the 15 selected products to identify gaps in the historical sales records. Missing dates were observed across several products, highlighting the need to consider data completeness before forecasting.

[35]:
```
selected_product_summary["date_coverage"] = (
    selected_product_summary["observations"]
    / selected_product_summary["calendar_days"]
)

coverage_check = selected_product_summary.sort_values(
    "date_coverage", ascending=False
)

display(
    coverage_check[
        [
            "observations",
            "first_date",
            "last_date",
            "calendar_days",
            "missing_dates",
            "date_coverage"
        ]
    ].round(3)
)

print(
    "Products with at least 90% date coverage:",
    (coverage_check["date_coverage"] >= 0.90).sum()
)
```

```
/tmp/ipykernel_607394/1545930987.py:20: UserWarning: obj.round has no effect
```

```
with datetime, timedelta, or period dtypes. Use obj.dt.round(…) instead.
  ].round(3)
```

unique_id	observations	first_date	last_date	calendar_days	missing_dates	date_coverage
4067	708	2022-06-24	2024-06-02	710	2	0.997
2792	998	2021-09-06	2024-06-02	1001	3	0.997
4425	997	2021-09-07	2024-06-02	1000	3	0.997
2723	1372	2020-08-01	2024-06-02	1402	30	0.979
4002	629	2022-08-24	2024-06-02	649	20	0.969
4345	1003	2021-08-02	2024-06-02	1036	33	0.968
2990	1352	2020-08-01	2024-05-30	1399	47	0.966
3788	457	2021-11-25	2023-04-03	495	38	0.923
2627	1223	2020-08-01	2024-06-02	1402	179	0.872
3000	1170	2020-08-01	2024-06-02	1402	232	0.835
3846	647	2022-03-14	2024-05-13	792	145	0.817
4361	742	2020-11-06	2023-06-25	962	220	0.771
4136	996	2020-08-01	2024-06-02	1402	406	0.710
3956	850	2020-08-01	2024-05-29	1398	548	0.608
2782	653	2020-08-01	2024-06-02	1402	749	0.466

```
Products with at least 90% date coverage: 8
```

The date coverage analysis showed that 8 out of the 15 selected products had at least 90% date coverage. The remaining products had larger gaps in their records. The 8 products meeting the coverage threshold were selected for further forecasting analysis.

[36]:
```
final_product_ids = coverage_check.loc[
    coverage_check["date_coverage"] >= 0.90
].index.tolist()
```

```
print("Final number of products:", len(final_product_ids))
print("Final product IDs:", final_product_ids)
```

```
Final number of products: 8
Final product IDs: [4067, 2792, 4425, 2723, 4002, 4345, 2990, 3788]
```

Based on the date coverage assessment, 8 products with at least 90% date coverage were selected for further analysis. This selection helps reduce the impact of missing dates during forecasting model evaluation.

[37]:
```
final_product_data = df_clean[
    df_clean["unique_id"].isin(final_product_ids)
].copy()

final_product_data = final_product_data.sort_values(
    ["unique_id", "date"]
)

print("Selected products:", final_product_data["unique_id"].nunique())
print("Total records:", len(final_product_data))
print("Date range:", final_product_data["date"].min(), "to",
      final_product_data["date"].max())

display(
    final_product_data.groupby("unique_id").agg(
        observations=("date", "nunique"),
        first_date=("date", "min"),
        last_date=("date", "max"),
        mean_sales=("sales", "mean")
    ).round(2)
)
```

```
Selected products: 8
Total records: 7516
Date range: 2020-08-01 00:00:00 to 2024-06-02 00:00:00
```

```
/tmp/ipykernel_607394/1127288607.py:20: UserWarning: obj.round has no effect
with datetime, timedelta, or period dtypes. Use obj.dt.round(…) instead.
  ).round(2)
```

unique_id	observations	first_date	last_date	mean_sales
2723	1372	2020-08-01	2024-06-02	16.62
2792	998	2021-09-06	2024-06-02	36.91
2990	1352	2020-08-01	2024-05-30	20.05
3788	457	2021-11-25	2023-04-03	48.05
4002	629	2022-08-24	2024-06-02	70.33
4067	708	2022-06-24	2024-06-02	48.87
4345	1003	2021-08-02	2024-06-02	66.82
4425	997	2021-09-07	2024-06-02	294.03

Preparing Data for Forecasting

The selected dataset contains 7,516 records across 8 products. The products have different observation counts and date ranges, so forecasting models will be evaluated separately for each product using chronological training and testing periods.

[38]:
```
coverage_summary = (
    final_product_data.groupby("unique_id")
    .agg(
        observations=("date", "nunique"),
        first_date=("date", "min"),
        last_date=("date", "max")
    )
)

coverage_summary["calendar_days"] = (
    coverage_summary["last_date"]
    - coverage_summary["first_date"]
).dt.days + 1

coverage_summary["missing_dates"] = (
    coverage_summary["calendar_days"]
    - coverage_summary["observations"]
)

display(
    coverage_summary[
        ["observations", "calendar_days", "missing_dates"]
    ]
)
```

unique_id	observations	calendar_days	missing_dates
2723	1372	1402	30
2792	998	1001	3
2990	1352	1399	47
3788	457	495	38
4002	629	649	20
4067	708	710	2
4345	1003	1036	33
4425	997	1000	3

Handling Missing Dates

The selected products contain some missing dates, ranging from 2 to 47 days. Since missing records do not necessarily indicate zero sales, the original observations are retained without automatically filling missing dates with zero values.

[39]:
```
gap_results = []

for product_id in final_product_ids:
    dates = (
        final_product_data.loc[
            final_product_data["unique_id"] == product_id, "date"
        ]
        .sort_values()
        .drop_duplicates()
    )

    full_dates = pd.date_range(dates.min(), dates.max(), freq="D")
    missing_dates = full_dates.difference(dates)

    if len(missing_dates) > 0:
        gap_series = pd.Series(missing_dates)
        gap_groups = gap_series.diff().dt.days.ne(1).cumsum()

        longest_gap = gap_series.groupby(gap_groups).size().max()
    else:
        longest_gap = 0

    gap_results.append({
        "unique_id": product_id,
        "missing_dates": len(missing_dates),
        "longest_consecutive_gap": longest_gap
    })

gap_summary = pd.DataFrame(gap_results).set_index("unique_id")
display(gap_summary)
```

unique_id	missing_dates	longest_consecutive_gap
4067	2	1
2792	3	1
4425	3	1
2723	30	9
4002	20	9
4345	33	5
2990	47	16
3788	38	7

Missing Date Pattern

The longest consecutive gap varied across products, ranging from 1 to 16 days. Products with longer gaps require careful handling during forecasting, as missing records cannot be assumed to represent zero sales.

[40]:
```
train_test_data = {}

for product_id in final_product_ids:
    product_df = (
        final_product_data[
            final_product_data["unique_id"] == product_id
        ]
        .sort_values("date")
        .reset_index(drop=True)
    )

    train_df = product_df.iloc[:-90].copy()
    test_df = product_df.iloc[-90:].copy()

    train_test_data[product_id] = {
        "train": train_df,
        "test": test_df
    }

    print(
        f"Product {product_id}: "
        f"Train = {len(train_df)}, Test = {len(test_df)}, "
        f"Test dates = {test_df['date'].min().date()} to "
        f"{test_df['date'].max().date()}"
    )
```

```
Product 4067: Train = 618, Test = 90, Test dates = 2024-03-05 to 2024-06-02
Product 2792: Train = 908, Test = 90, Test dates = 2024-03-05 to 2024-06-02
Product 4425: Train = 907, Test = 90, Test dates = 2024-03-05 to 2024-06-02
Product 2723: Train = 1282, Test = 90, Test dates = 2024-03-05 to 2024-06-02
Product 4002: Train = 539, Test = 90, Test dates = 2024-03-05 to 2024-06-02
Product 4345: Train = 913, Test = 90, Test dates = 2024-02-25 to 2024-06-02
Product 2990: Train = 1262, Test = 90, Test dates = 2024-03-01 to 2024-05-30
Product 3788: Train = 367, Test = 90, Test dates = 2022-12-26 to 2023-04-03
```

Training and Testing Data

A common testing period from 5 March 2024 to 2 June 2024 was selected to improve the fairness of model comparison. Product 3788 was excluded because its historical records end in April 2023. Missing dates within the testing period are retained as missing rather than treated as zero sales.

[41]:
```
from datetime import datetime

eval_product_ids = [
    product_id for product_id in final_product_ids
    if product_id != 3788
]

test_start = pd.Timestamp("2024-03-05")
```

```
test_end = pd.Timestamp("2024-06-02")

common_period_data = {}

for product_id in eval_product_ids:
    product_df = (
        final_product_data[
            final_product_data["unique_id"] == product_id
        ]
        .sort_values("date")
        .reset_index(drop=True)
    )

    train_df = product_df[product_df["date"] < test_start].copy()
    test_df = product_df[
        (product_df["date"] >= test_start) &
        (product_df["date"] <= test_end)
    ].copy()

    common_period_data[product_id] = {
        "train": train_df,
        "test": test_df
    }

    print(
        f"Product {product_id}: "
        f"Train = {len(train_df)}, "
        f"Test observations = {len(test_df)}, "
        f"Missing test dates = {90 - len(test_df)}"
    )
```

```
Product 4067: Train = 618, Test observations = 90, Missing test dates = 0
Product 2792: Train = 908, Test observations = 90, Missing test dates = 0
Product 4425: Train = 907, Test observations = 90, Missing test dates = 0
Product 2723: Train = 1282, Test observations = 90, Missing test dates = 0
Product 4002: Train = 539, Test observations = 90, Missing test dates = 0
Product 4345: Train = 922, Test observations = 81, Missing test dates = 9
Product 2990: Train = 1266, Test observations = 86, Missing test dates = 4
```

Moving Average Forecast

A 7-observation moving average was used as the baseline forecasting model. The average sales of the last seven training observations was used as the forecast for the testing period. Model performance was evaluated using MAE, RMSE and MAPE.

[42]:
```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

```
baseline_results = []

for product_id, data in common_period_data.items():
    train_df = data["train"]
    test_df = data["test"]

    forecast_value = train_df["sales"].tail(7).mean()

    actual = test_df["sales"].to_numpy()
    predicted = np.full(len(actual), forecast_value)

    mae = mean_absolute_error(actual, predicted)
    rmse = np.sqrt(mean_squared_error(actual, predicted))

    nonzero_mask = actual != 0
    mape = (
        np.mean(
            np.abs(
                (actual[nonzero_mask] - predicted[nonzero_mask])
                / actual[nonzero_mask]
            )
        ) * 100
        if nonzero_mask.any()
        else np.nan
    )

    baseline_results.append({
        "unique_id": product_id,
        "forecast_value": forecast_value,
        "test_observations": len(actual),
        "MAE": mae,
        "RMSE": rmse,
        "MAPE_percent": mape
    })

baseline_results_df = pd.DataFrame(baseline_results)

display(baseline_results_df.round(2))
```

	unique_id	forecast_value	test_observations	MAE	RMSE	MAPE_percent
0	4067	59.77	90	17.13	22.17	27.17
1	2792	26.78	90	5.73	7.46	25.28
2	4425	268.73	90	43.76	56.28	16.55
3	2723	10.67	90	4.51	6.79	32.47
4	4002	93.01	90	33.98	43.09	60.26
5	4345	67.48	81	55.22	68.59	41.98
6	2990	21.70	86	4.94	6.96	16.62

Simple Exponential Smoothing

Simple Exponential Smoothing (SES) was applied to the selected products as another forecasting method. The model assigns greater weight to recent observations and produces forecasts for the testing period. Its performance was evaluated using MAE, RMSE and MAPE.

[43]:
```
from statsmodels.tsa.holtwinters import SimpleExpSmoothing

ses_results = []

for product_id, data in common_period_data.items():
    train_df = data["train"]
    test_df = data["test"]

    model = SimpleExpSmoothing(
        train_df["sales"].to_numpy(),
        initialization_method="estimated"
    ).fit()

    predictions = model.forecast(len(test_df))
    actual = test_df["sales"].to_numpy()

    mae = mean_absolute_error(actual, predictions)
    rmse = np.sqrt(mean_squared_error(actual, predictions))

    nonzero_mask = actual != 0
    mape = (
        np.mean(
            np.abs(
                (actual[nonzero_mask] - predictions[nonzero_mask])
                / actual[nonzero_mask]
            )
        ) * 100
        if nonzero_mask.any()
        else np.nan
    )

    ses_results.append({
        "unique_id": product_id,
        "test_observations": len(actual),
        "MAE": mae,
        "RMSE": rmse,
        "MAPE_percent": mape
    })

ses_results_df = pd.DataFrame(ses_results)

display(ses_results_df.round(2))
```

```
   unique_id  test_observations    MAE   RMSE  MAPE_percent
0       4067                 90  17.07  22.09         27.18
1       2792                 90   5.74   7.47         25.50
2       4425                 90  42.94  55.10         17.22
3       2723                 90   5.09   6.53         48.87
4       4002                 90  33.30  42.95         57.54
5       4345                 81  56.06  69.43         42.53
6       2990                 86   4.11   6.05         14.03
```

Comparison of Moving Average and SES

The comparison showed that neither model performed best across all products. Simple Exponential Smoothing achieved lower MAE for four products, while the Moving Average model performed better for the other three. This indicates that forecasting performance varies across products.

[44]:
```
comparison_df = baseline_results_df[
    ["unique_id", "MAE", "RMSE", "MAPE_percent"]
].merge(
    ses_results_df[
        ["unique_id", "MAE", "RMSE", "MAPE_percent"]
    ],
    on="unique_id",
    suffixes=("_Moving_Average", "_SES")
)

comparison_df["Better_model_by_MAE"] = np.where(
    comparison_df["MAE_Moving_Average"] < comparison_df["MAE_SES"],
    "Moving Average",
    np.where(
        comparison_df["MAE_SES"] < comparison_df["MAE_Moving_Average"],
        "SES",
        "Tie"
    )
)

display(comparison_df.round(2))
```

```
   unique_id  MAE_Moving_Average  RMSE_Moving_Average  \
0       4067               17.13                22.17
1       2792                5.73                 7.46
2       4425               43.76                56.28
3       2723                4.51                 6.79
4       4002               33.98                43.09
5       4345               55.22                68.59
6       2990                4.94                 6.96

   MAPE_percent_Moving_Average  MAE_SES  RMSE_SES  MAPE_percent_SES  \
0                        27.17    17.07     22.09             27.18
1                        25.28     5.74      7.47             25.50
```

```
2                      16.55    42.94     55.10             17.22
3                      32.47     5.09      6.53             48.87
4                      60.26    33.30     42.95             57.54
5                      41.98    56.06     69.43             42.53
6                      16.62     4.11      6.05             14.03

  Better_model_by_MAE
0                 SES
1      Moving Average
2                 SES
3      Moving Average
4                 SES
5      Moving Average
6                 SES
```

Moving Average and SES Model Comparison

The two models showed different performance across the selected products. SES achieved lower MAE for four products, while Moving Average performed better for three products. The differences were generally small for some products, so additional forecasting models were evaluated for comparison.

[45]:
```python
from statsmodels.tsa.holtwinters import Holt

holt_results = []

for product_id, data in common_period_data.items():
    train_df = data["train"]
    test_df = data["test"]

    model = Holt(
        train_df["sales"].to_numpy(),
        initialization_method="estimated"
    ).fit(optimized=True)

    predictions = model.forecast(len(test_df))
    actual = test_df["sales"].to_numpy()

    mae = mean_absolute_error(actual, predictions)
    rmse = np.sqrt(mean_squared_error(actual, predictions))

    nonzero_mask = actual != 0
    mape = (
        np.mean(
            np.abs(
                (actual[nonzero_mask] - predictions[nonzero_mask])
                / actual[nonzero_mask]
            )
```

```
        ) * 100
        if nonzero_mask.any()
        else np.nan
    )

    holt_results.append({
        "unique_id": product_id,
        "test_observations": len(actual),
        "MAE": mae,
        "RMSE": rmse,
        "MAPE_percent": mape
    })

holt_results_df = pd.DataFrame(holt_results)

display(holt_results_df.round(2))
```

	unique_id	test_observations	MAE	RMSE	MAPE_percent
0	4067	90	16.08	21.09	27.97
1	2792	90	5.73	7.52	24.33
2	4425	90	43.87	56.11	16.89
3	2723	90	5.19	6.58	50.30
4	4002	90	34.78	43.64	62.48
5	4345	81	53.75	66.92	41.13
6	2990	86	3.95	5.83	13.66

0.1.1 Holt's Linear Trend Model

Holt's Linear Trend model was evaluated to examine whether incorporating a trend component improves forecasting accuracy. The results varied across products, indicating that adding a trend component does not consistently improve prediction accuracy.

[46]:
```
holt_comparison = (
    baseline_results_df[["unique_id", "MAE", "RMSE", "MAPE_percent"]]
    .merge(
        ses_results_df[["unique_id", "MAE", "RMSE", "MAPE_percent"]],
        on="unique_id",
        suffixes=("_MA", "_SES")
    )
    .merge(
        holt_results_df[["unique_id", "MAE", "RMSE", "MAPE_percent"]],
        on="unique_id"
    )
    .rename(columns={
        "MAE": "MAE_Holt",
        "RMSE": "RMSE_Holt",
        "MAPE_percent": "MAPE_Holt"
    })
```

```
)

mae_columns = ["MAE_MA", "MAE_SES", "MAE_Holt"]

holt_comparison["Best_model_by_MAE"] = (
    holt_comparison[mae_columns]
    .idxmin(axis=1)
    .map({
        "MAE_MA": "Moving Average",
        "MAE_SES": "SES",
        "MAE_Holt": "Holt Trend"
    })
)

display(holt_comparison.round(2))
```

```
   unique_id  MAE_MA  RMSE_MA  MAPE_percent_MA  MAE_SES  RMSE_SES  \
0       4067   17.13    22.17            27.17    17.07     22.09
1       2792    5.73     7.46            25.28     5.74      7.47
2       4425   43.76    56.28            16.55    42.94     55.10
3       2723    4.51     6.79            32.47     5.09      6.53
4       4002   33.98    43.09            60.26    33.30     42.95
5       4345   55.22    68.59            41.98    56.06     69.43
6       2990    4.94     6.96            16.62     4.11      6.05

   MAPE_percent_SES  MAE_Holt  RMSE_Holt  MAPE_Holt Best_model_by_MAE
0             27.18     16.08      21.09      27.97        Holt Trend
1             25.50      5.73       7.52      24.33        Holt Trend
2             17.22     43.87      56.11      16.89               SES
3             48.87      5.19       6.58      50.30    Moving Average
4             57.54     34.78      43.64      62.48               SES
5             42.53     53.75      66.92      41.13        Holt Trend
6             14.03      3.95       5.83      13.66        Holt Trend
```

0.1.2 Comparison of Forecasting Models

The three forecasting models were compared using MAE, RMSE and MAPE. Based on MAE, Holt's Linear Trend model performed best for four products, Simple Exponential Smoothing performed best for two products, and the Moving Average model performed best for one product. These results indicate that forecasting performance varies across products and that no single model is consistently superior for all products.

[47]:
```
overall_model_comparison = pd.DataFrame({
    "Model": ["Moving Average", "SES", "Holt Trend"],
    "Mean_MAE": [
        baseline_results_df["MAE"].mean(),
        ses_results_df["MAE"].mean(),
        holt_results_df["MAE"].mean()
```

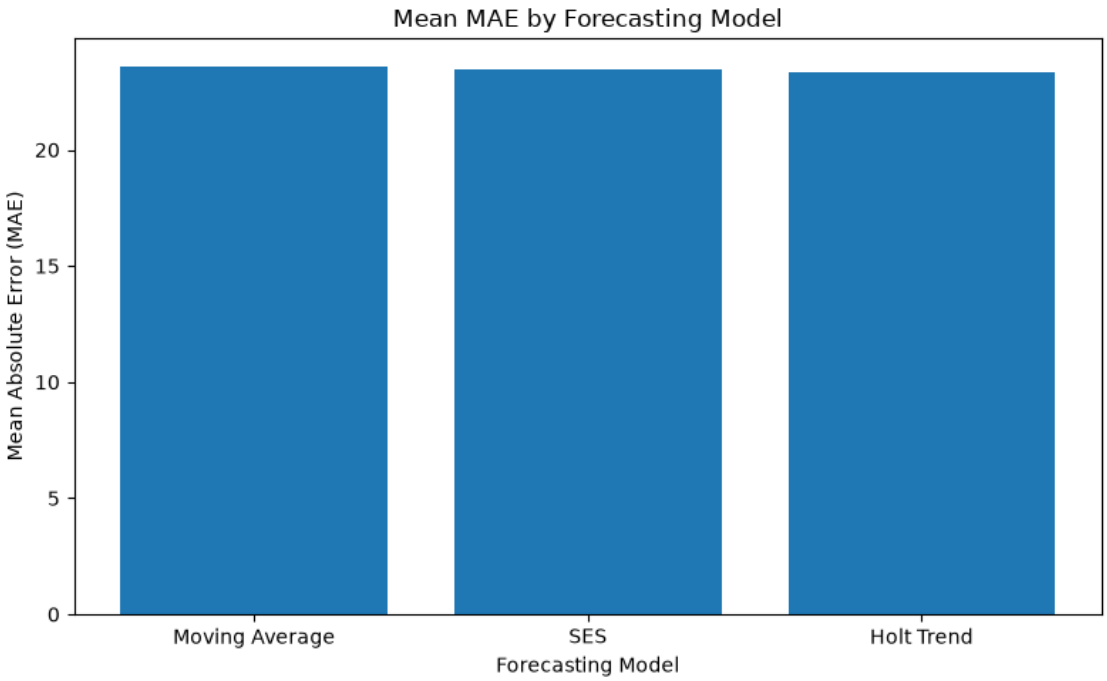
```
    ],
    "Mean_RMSE": [
        baseline_results_df["RMSE"].mean(),
        ses_results_df["RMSE"].mean(),
        holt_results_df["RMSE"].mean()
    ],
    "Mean_MAPE_percent": [
        baseline_results_df["MAPE_percent"].mean(),
        ses_results_df["MAPE_percent"].mean(),
        holt_results_df["MAPE_percent"].mean()
    ]
})

display(overall_model_comparison.round(2))
```

	Model	Mean_MAE	Mean_RMSE	Mean_MAPE_percent
0	Moving Average	23.61	30.19	31.48
1	SES	23.47	29.95	33.27
2	Holt Trend	23.34	29.67	33.82

[48]:
```
import matplotlib.pyplot as plt

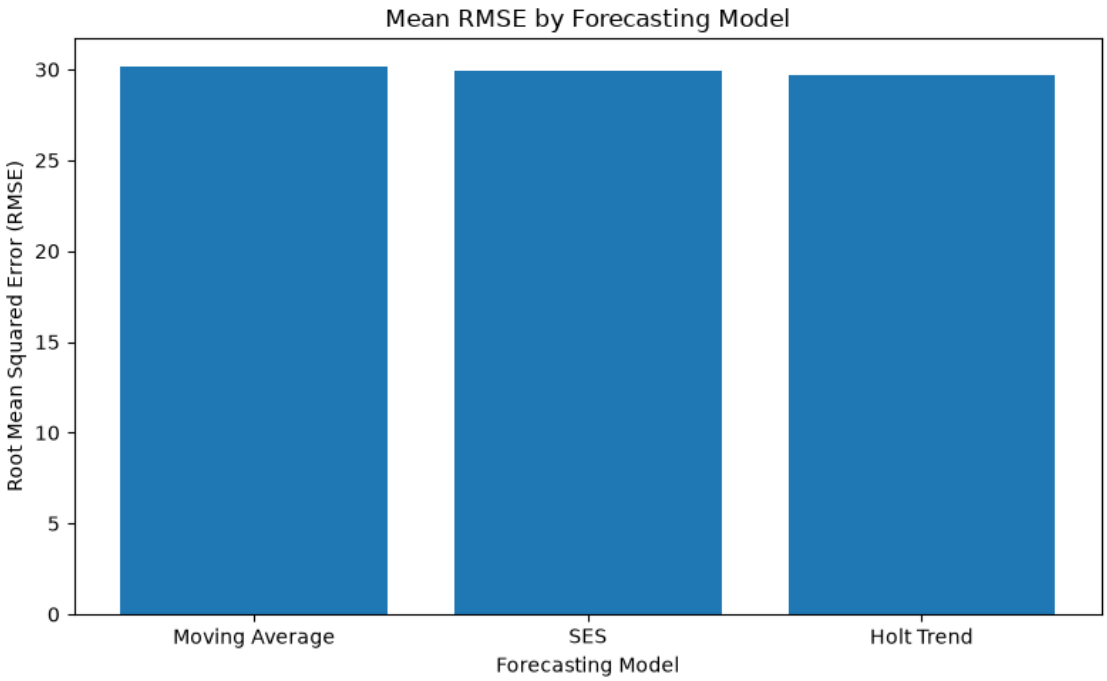
plt.figure(figsize=(8, 5))
plt.bar(
    overall_model_comparison["Model"],
    overall_model_comparison["Mean_MAE"]
)
plt.title("Mean MAE by Forecasting Model")
plt.xlabel("Forecasting Model")
plt.ylabel("Mean Absolute Error (MAE)")
plt.tight_layout()
plt.show()
```



0.1.3 Visual Comparison of Model Performance

The mean MAE values were very similar across the three forecasting models. Holt's Linear Trend model recorded the lowest mean MAE, but the difference from the other models was small. This suggests that the choice of forecasting model may depend more on individual product characteristics than on overall average performance.

[49]:
```
plt.figure(figsize=(8, 5))
plt.bar(
    overall_model_comparison["Model"],
    overall_model_comparison["Mean_RMSE"]
)
plt.title("Mean RMSE by Forecasting Model")
plt.xlabel("Forecasting Model")
plt.ylabel("Root Mean Squared Error (RMSE)")
plt.tight_layout()
plt.show()
```



0.1.4 Interpretation of Overall Model Performance

The visual comparison of mean MAE and RMSE showed similar error levels across all three forecasting models. Although Holt's Linear Trend model achieved slightly lower average errors, the differences were small. Therefore, product-level performance and the percentage error metric are also considered when evaluating the forecasting methods.

[50]:
```
best_models_by_metric = pd.DataFrame({
    "Metric": ["Mean MAE", "Mean RMSE", "Mean MAPE"],
    "Best Model": [
        overall_model_comparison.loc[
            overall_model_comparison["Mean_MAE"].idxmin(), "Model"
        ],
        overall_model_comparison.loc[
            overall_model_comparison["Mean_RMSE"].idxmin(), "Model"
        ],
        overall_model_comparison.loc[
            overall_model_comparison["Mean_MAPE_percent"].idxmin(), "Model"
        ]
    ]
})

display(best_models_by_metric)
```

```
      Metric      Best Model
0   Mean MAE      Holt Trend
```

```
1  Mean RMSE      Holt Trend
2  Mean MAPE  Moving Average
```

0.1.5 Best Model by Evaluation Metric

Holt's Linear Trend model achieved the lowest mean MAE and RMSE, whereas the Moving Average model achieved the lowest mean MAPE. The results show that the preferred model depends on the evaluation metric used. Since the differences in average errors are small, the product-level results should also be considered before drawing conclusions.

[51]:
```
zero_sales_summary = []

for product_id, data in common_period_data.items():
    test_df = data["test"]

    zero_sales_summary.append({
        "unique_id": product_id,
        "test_observations": len(test_df),
        "zero_sales_days": int((test_df["sales"] == 0).sum()),
        "zero_sales_percent": (test_df["sales"] == 0).mean() * 100
    })

zero_sales_summary_df = pd.DataFrame(zero_sales_summary)
display(zero_sales_summary_df.round(2))
```

	unique_id	test_observations	zero_sales_days	zero_sales_percent
0	4067	90	0	0.0
1	2792	90	0	0.0
2	4425	90	0	0.0
3	2723	90	0	0.0
4	4002	90	0	0.0
5	4345	81	0	0.0
6	2990	86	0	0.0

0.1.6 Zero-Sales Observation Check

The testing data was examined for zero-sales observations because MAPE is undefined when actual sales are zero. No zero-sales observations were found among the recorded testing data for the seven products. Therefore, all observed testing records were included in the MAPE calculation. However, missing dates remain a separate data limitation and were not treated as zero sales.

[52]:
```
test_coverage_summary = []

for product_id, data in common_period_data.items():
    test_df = data["test"]

    calendar_days = (test_end - test_start).days + 1

    test_coverage_summary.append({
```

```
        "unique_id": product_id,
        "first_test_date": test_df["date"].min().date(),
        "last_test_date": test_df["date"].max().date(),
        "observed_test_records": len(test_df),
        "missing_dates_in_test_period": calendar_days - test_df["date"].
↪nunique()
    })

test_coverage_summary_df = pd.DataFrame(test_coverage_summary)
display(test_coverage_summary_df)
```

```
   unique_id first_test_date last_test_date  observed_test_records  \
0       4067      2024-03-05     2024-06-02                     90
1       2792      2024-03-05     2024-06-02                     90
2       4425      2024-03-05     2024-06-02                     90
3       2723      2024-03-05     2024-06-02                     90
4       4002      2024-03-05     2024-06-02                     90
5       4345      2024-03-05     2024-06-02                     81
6       2990      2024-03-05     2024-05-30                     86

   missing_dates_in_test_period
0                             0
1                             0
2                             0
3                             0
4                             0
5                             9
6                             4
```

0.1.7 Testing Period Data Coverage

The testing period covers 5 March 2024 to 2 June 2024. Five products had complete records for all 90 calendar days, while product 4345 had 9 missing dates and product 2990 had 4 missing dates. These missing records were not assumed to represent zero sales. This limitation was considered when interpreting the forecasting results.

[53]:
```
product_id = 4345

train_df = common_period_data[product_id]["train"].sort_values("date")
test_df = common_period_data[product_id]["test"].sort_values("date")

alignment_check = pd.DataFrame({
    "actual_date": test_df["date"].reset_index(drop=True),
    "actual_sales": test_df["sales"].reset_index(drop=True)
})

alignment_check["forecast_step"] = range(1, len(alignment_check) + 1)
```

```
display(alignment_check.head(15))
```

	actual_date	actual_sales	forecast_step
0	2024-03-05	115.30	1
1	2024-03-06	89.85	2
2	2024-03-07	129.37	3
3	2024-03-08	135.97	4
4	2024-03-11	67.49	5
5	2024-03-12	116.56	6
6	2024-03-13	107.90	7
7	2024-03-14	81.24	8
8	2024-03-15	161.32	9
9	2024-03-16	73.71	10
10	2024-03-17	63.20	11
11	2024-03-18	67.28	12
12	2024-03-19	94.72	13
13	2024-03-20	112.13	14
14	2024-03-21	138.19	15

0.1.8 Aligning Forecasts with Calendar Dates

Some products contain missing dates within the testing period. To maintain a consistent evaluation, forecasts should be aligned with the actual calendar dates rather than only the order of observed records. Missing sales values will remain unknown instead of being replaced with zero, since absent records do not necessarily indicate zero demand.

[54]:
```
training_gap_summary = []

for product_id, data in common_period_data.items():
    train_df = data["train"].sort_values("date")

    calendar_days = (
        train_df["date"].max() - train_df["date"].min()
    ).days + 1

    observed_days = train_df["date"].nunique()

    training_gap_summary.append({
        "unique_id": product_id,
        "training_start": train_df["date"].min().date(),
        "training_end": train_df["date"].max().date(),
        "observed_days": observed_days,
        "missing_calendar_dates": calendar_days - observed_days
    })

training_gap_summary_df = pd.DataFrame(training_gap_summary)
display(training_gap_summary_df)
```

```
   unique_id training_start training_end  observed_days  \
0       4067     2022-06-24   2024-03-04            618
1       2792     2021-09-06   2024-03-04            908
2       4425     2021-09-07   2024-03-04            907
3       2723     2020-08-01   2024-03-04           1282
4       4002     2022-08-24   2024-03-04            539
5       4345     2021-08-02   2024-03-04            922
6       2990     2020-08-01   2024-03-04           1266

   missing_calendar_dates
0                       2
1                       3
2                       3
3                      30
4                      20
5                      24
6                      46
```

0.1.9 Preparing a Consistent Daily Time Series

The training and testing periods contain missing calendar dates for some products. A daily date index will be used to identify these gaps explicitly. Missing sales values will remain unknown unless there is evidence that they represent zero sales. Forecast evaluation will use matching calendar dates to ensure that predictions and actual sales are compared consistently.

[55]:
```
product_id = 4345

product_df = (
    final_product_data[final_product_data["unique_id"] == product_id]
    .sort_values("date")
    .set_index("date")
)

daily_product = product_df[["sales"]].asfreq("D")

print("Original observed records:", len(product_df))
print("Daily calendar records:", len(daily_product))
print("Missing sales after daily reindexing:", daily_product["sales"].isna().
  ↪sum())

display(daily_product.loc["2024-03-05":"2024-03-15"])
```

```
Original observed records: 1003
Daily calendar records: 1036
Missing sales after daily reindexing: 33

             sales
date
2024-03-05  115.30
```

```
2024-03-06   89.85
2024-03-07  129.37
2024-03-08  135.97
2024-03-09     NaN
2024-03-10     NaN
2024-03-11   67.49
2024-03-12  116.56
2024-03-13  107.90
2024-03-14   81.24
2024-03-15  161.32
```

0.1.10 Daily Date Alignment Check

Reindexing product 4345 to a daily calendar identified 33 dates without recorded sales values. These missing values were retained as unknown rather than replaced with zero. This check highlights the importance of accounting for missing dates when preparing time-series data and evaluating forecasting accuracy.

[56]:
```
product_id = 4345

train_df = common_period_data[product_id]["train"].sort_values("date")
test_df = common_period_data[product_id]["test"].sort_values("date")

print("Last training date:", train_df["date"].max())
print("First test date:", test_df["date"].min())
print("Last test date:", test_df["date"].max())
print("Training records:", len(train_df))
print("Testing records:", len(test_df))
print("Missing calendar dates in test period:",
      ((test_end - test_start).days + 1) - test_df["date"].nunique())
```

```
Last training date: 2024-03-04 00:00:00
First test date: 2024-03-05 00:00:00
Last test date: 2024-06-02 00:00:00
Training records: 922
Testing records: 81
Missing calendar dates in test period: 9
```

0.1.11 Validation of the Training and Testing Split

The training and testing periods are chronologically separated, with training ending on 4 March 2024 and testing beginning on 5 March 2024. However, the testing data for product 4345 contains 9 missing calendar dates. Forecasts must therefore be aligned with calendar dates before the final accuracy comparison is reported.

[57]:
```
gap_validation = []

for product_id, data in common_period_data.items():
    train_df = data["train"]
```

```
    test_df = data["test"]

    train_calendar_days = (
        train_df["date"].max() - train_df["date"].min()
    ).days + 1

    test_calendar_days = (test_end - test_start).days + 1

    gap_validation.append({
        "unique_id": product_id,
        "training_missing_dates": train_calendar_days - train_df["date"].
↪nunique(),
        "testing_missing_dates": test_calendar_days - test_df["date"].nunique()
    })

gap_validation_df = pd.DataFrame(gap_validation)
display(gap_validation_df)
```

	unique_id	training_missing_dates	testing_missing_dates
0	4067	2	0
1	2792	3	0
2	4425	3	0
3	2723	30	0
4	4002	20	0
5	4345	24	9
6	2990	46	4

0.1.12 Summary of Missing-Date Validation

Missing calendar dates were identified in both training and testing periods. Five products had complete observations throughout the testing period, while products 4345 and 2990 had 9 and 4 missing dates, respectively. The existing model results are retained as preliminary findings. A calendar-aligned evaluation is required before finalising the comparison across forecasting methods.

[58]:
```
import inspect
from statsmodels.tsa.holtwinters import (
    SimpleExpSmoothing, Holt
)

print("SES fit parameters:")
print(inspect.signature(SimpleExpSmoothing.fit))

print("\nHolt fit parameters:")
print(inspect.signature(Holt.fit))
```

```
SES fit parameters:
(self, smoothing_level=None, *, optimized=True, start_params=None,
use_brute=True, remove_bias=False, method=None, minimize_kwargs=None)
```

```
Holt fit parameters:
(self, smoothing_level=None, smoothing_trend=None, *, damping_trend=None,
optimized=True, start_params=None, use_brute=True, remove_bias=False,
method=None, minimize_kwargs=None)
```

0.1.13 Evaluation Approach and Data Limitations

The initial forecasting results are treated as preliminary because missing dates may affect the time intervals represented by successive observations. For a consistent test-period comparison, the five products with complete daily testing records will be evaluated separately. Missing training dates will remain an acknowledged limitation, and absent sales records will not be assumed to represent zero demand.

[59]:
```
complete_test_product_ids = gap_validation_df.loc[
    gap_validation_df["testing_missing_dates"] == 0,
    "unique_id"
].tolist()

print("Products with complete test coverage:", complete_test_product_ids)
print("Number of products:", len(complete_test_product_ids))
```

```
Products with complete test coverage: [4067, 2792, 4425, 2723, 4002]
Number of products: 5
```

0.1.14 Final Evaluation Sample

Five products (4067, 2792, 4425, 2723 and 4002) have complete observations throughout the 90-day testing period from 5 March to 2 June 2024. These products were selected for the next stage of model evaluation to ensure that actual sales are available for every calendar day in the testing period. The two products with missing test dates will remain part of the preliminary analysis but will be excluded from this comparison.

[60]:
```
complete_test_comparison = (
    baseline_results_df[
        baseline_results_df["unique_id"].isin(complete_test_product_ids)
    ][["unique_id", "MAE", "RMSE", "MAPE_percent"]]
    .merge(
        ses_results_df[
            ses_results_df["unique_id"].isin(complete_test_product_ids)
        ][["unique_id", "MAE", "RMSE", "MAPE_percent"]],
        on="unique_id",
        suffixes=("_MA", "_SES")
    )
    .merge(
        holt_results_df[
            holt_results_df["unique_id"].isin(complete_test_product_ids)
        ][["unique_id", "MAE", "RMSE", "MAPE_percent"]],
        on="unique_id"
    )
```

```
    .rename(columns={
        "MAE": "MAE_Holt",
        "RMSE": "RMSE_Holt",
        "MAPE_percent": "MAPE_Holt"
    })
)

display(complete_test_comparison.round(2))
```

	unique_id	MAE_MA	RMSE_MA	MAPE_percent_MA	MAE_SES	RMSE_SES	\
0	4067	17.13	22.17	27.17	17.07	22.09	
1	2792	5.73	7.46	25.28	5.74	7.47	
2	4425	43.76	56.28	16.55	42.94	55.10	
3	2723	4.51	6.79	32.47	5.09	6.53	
4	4002	33.98	43.09	60.26	33.30	42.95	

	MAPE_percent_SES	MAE_Holt	RMSE_Holt	MAPE_Holt
0	27.18	16.08	21.09	27.97
1	25.50	5.73	7.52	24.33
2	17.22	43.87	56.11	16.89
3	48.87	5.19	6.58	50.30
4	57.54	34.78	43.64	62.48

0.1.15 Product-Level Model Comparison

The five products with complete test-period coverage showed different levels of forecasting accuracy across the three models. Moving Average performed best for product 2723, while SES performed best for products 4425 and 4002. Holt Trend achieved the lowest MAE for product 4067, and the results for product 2792 were very close across models. These findings suggest that forecasting performance varies by product.

[61]:
```
complete_test_overall = pd.DataFrame({
    "Model": ["Moving Average", "SES", "Holt Trend"],
    "Mean_MAE": [
        complete_test_comparison["MAE_MA"].mean(),
        complete_test_comparison["MAE_SES"].mean(),
        complete_test_comparison["MAE_Holt"].mean()
    ],
    "Mean_RMSE": [
        complete_test_comparison["RMSE_MA"].mean(),
        complete_test_comparison["RMSE_SES"].mean(),
        complete_test_comparison["RMSE_Holt"].mean()
    ],
    "Mean_MAPE_percent": [
        complete_test_comparison["MAPE_percent_MA"].mean(),
        complete_test_comparison["MAPE_percent_SES"].mean(),
        complete_test_comparison["MAPE_Holt"].mean()
    ]
```

```
})

display(complete_test_overall.round(2))
```

	Model	Mean_MAE	Mean_RMSE	Mean_MAPE_percent
0	Moving Average	21.02	27.16	32.35
1	SES	20.83	26.83	35.26
2	Holt Trend	21.13	26.99	36.39

0.1.16 Overall Performance on Products with Complete Test Coverage

For the five products with complete testing records, Simple Exponential Smoothing achieved the lowest mean MAE and RMSE, while the Moving Average model achieved the lowest mean MAPE. The differences between the models were small, indicating that no model demonstrated a clear advantage across all evaluation metrics. Missing dates in the training data remain a limitation when interpreting these preliminary results.

[62]:
```
training_gap_validation = []

for product_id in complete_test_product_ids:
    train_df = common_period_data[product_id]["train"].sort_values("date")

    calendar_days = (
        train_df["date"].max() - train_df["date"].min()
    ).days + 1

    missing_dates = calendar_days - train_df["date"].nunique()

    training_gap_validation.append({
        "unique_id": product_id,
        "calendar_days": calendar_days,
        "observed_records": train_df["date"].nunique(),
        "missing_dates": missing_dates,
        "missing_percent": missing_dates / calendar_days * 100
    })

training_gap_validation_df = pd.DataFrame(training_gap_validation)
display(training_gap_validation_df.round(2))
```

	unique_id	calendar_days	observed_records	missing_dates	missing_percent
0	4067	620	618	2	0.32
1	2792	911	908	3	0.33
2	4425	910	907	3	0.33
3	2723	1312	1282	30	2.29
4	4002	559	539	20	3.58

0.1.17 Training Data Completeness

The five selected products had between 0.32% and 3.58% missing calendar dates in their training periods. Products 4067, 2792 and 4425 had very high date coverage, while products 2723 and 4002 had slightly more gaps. Although the missing-date percentages were relatively low, their effect on forecasting accuracy should be considered before final conclusions are drawn.

[63]:
```
product_id = 4067

train_df = common_period_data[product_id]["train"].sort_values("date")
test_df = common_period_data[product_id]["test"].sort_values("date")

train_daily = (
    train_df.set_index("date")[["sales"]]
    .asfreq("D")
)

train_daily["sales"] = train_daily["sales"].interpolate(method="time")

print("Training calendar days:", len(train_daily))
print("Missing training values after interpolation:",
      train_daily["sales"].isna().sum())
print("Observed test records:", len(test_df))
print("Missing test sales:", test_df["sales"].isna().sum())
```

```
Training calendar days: 620
Missing training values after interpolation: 0
Observed test records: 90
Missing test sales: 0
```

0.1.18 Handling Missing Training Values

For product 4067, the training data was converted to a daily time series. The two missing training values were estimated using time-based linear interpolation, while the testing data was left unchanged. This approach provides a continuous training series for the trial evaluation. Interpolated values are estimates and may differ from actual sales.

[64]:
```
from statsmodels.tsa.holtwinters import (
    SimpleExpSmoothing, Holt
)
from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

train_values = train_daily["sales"].to_numpy()
actual = test_df["sales"].to_numpy()
actual_dates = pd.DatetimeIndex(test_df["date"])
forecast_dates = pd.date_range(test_start, test_end, freq="D")
forecast_positions = forecast_dates.get_indexer(actual_dates)
if (forecast_positions < 0).any():
```

```
    raise ValueError("An observed test date falls outside the forecast␣
 ↪calendar")
nonzero_mask = actual != 0

model_predictions = {
    "Moving Average": np.full(len(forecast_dates), train_daily["sales"].tail(7).
 ↪mean()),
    "SES": SimpleExpSmoothing(
        train_values, initialization_method="estimated"
    ).fit().forecast(len(forecast_dates)),
    "Holt Trend": Holt(
        train_values, initialization_method="estimated"
    ).fit(optimized=True).forecast(len(forecast_dates))
}

model_predictions = {
    model_name: np.asarray(predictions)[forecast_positions]
    for model_name, predictions in model_predictions.items()
}

corrected_results = []

for model_name, predictions in model_predictions.items():
    corrected_results.append({
        "Model": model_name,
        "MAE": mean_absolute_error(actual, predictions),
        "RMSE": np.sqrt(mean_squared_error(actual, predictions)),
        "MAPE_percent": (
            np.mean(np.abs((actual[nonzero_mask] - np.
 ↪asarray(predictions)[nonzero_mask]) / actual[nonzero_mask])) * 100
            if nonzero_mask.any() else np.nan
        ),
        "MAPE_observations": int(nonzero_mask.sum()),
        "Zero_actuals_excluded_from_MAPE": int((~nonzero_mask).sum())
    })

corrected_product_4067 = pd.DataFrame(corrected_results)
display(corrected_product_4067.round(2))
```

```
            Model    MAE   RMSE  MAPE_percent  MAPE_observations  \
0  Moving Average  17.13  22.17         27.17                 90
1             SES  17.07  22.09         27.18                 90
2      Holt Trend  16.08  21.10         27.95                 90

   Zero_actuals_excluded_from_MAPE
0                                0
1                                0
```

2 0

0.1.19 Corrected Evaluation Trial: Product 4067

The models were re-evaluated after converting the training data to daily frequency and interpolating two missing training values. The resulting MAE and RMSE values were unchanged to two decimal places, while MAPE showed only a negligible difference for the Moving Average and SES models. This suggests that the two missing training values had little effect on this product's results. Further products require separate evaluation before drawing a general conclusion.

[65]:
```
product_id = 2792

train_df = common_period_data[product_id]["train"].sort_values("date")
test_df = common_period_data[product_id]["test"].sort_values("date")

train_daily_2792 = (
    train_df.set_index("date")[["sales"]]
    .asfreq("D")
)

train_daily_2792["sales"] = (
    train_daily_2792["sales"].interpolate(method="time")
)

print("Training calendar days:", len(train_daily_2792))
print("Missing training values after interpolation:",
      train_daily_2792["sales"].isna().sum())
print("Test records:", len(test_df))
print("Missing test sales:", test_df["sales"].isna().sum())
```

```
Training calendar days: 911
Missing training values after interpolation: 0
Test records: 90
Missing test sales: 0
```

0.1.20 Missing-Value Treatment: Product 2792

For product 2792, the training series was converted to daily frequency and three missing training values were estimated using time-based linear interpolation. The testing data contains 90 observed records and was not modified. The models can now be evaluated using the daily training series and the original test observations.

[66]:
```
train_values = train_daily_2792["sales"].to_numpy()
actual = test_df["sales"].to_numpy()
actual_dates = pd.DatetimeIndex(test_df["date"])
forecast_dates = pd.date_range(test_start, test_end, freq="D")
forecast_positions = forecast_dates.get_indexer(actual_dates)
if (forecast_positions < 0).any():
```

```
    raise ValueError("An observed test date falls outside the forecast␣
 ↪calendar")
nonzero_mask = actual != 0

model_predictions = {
    "Moving Average": np.full(
        len(forecast_dates), train_daily_2792["sales"].tail(7).mean()
    ),
    "SES": SimpleExpSmoothing(
        train_values, initialization_method="estimated"
    ).fit().forecast(len(forecast_dates)),
    "Holt Trend": Holt(
        train_values, initialization_method="estimated"
    ).fit(optimized=True).forecast(len(forecast_dates))
}

model_predictions = {
    model_name: np.asarray(predictions)[forecast_positions]
    for model_name, predictions in model_predictions.items()
}

corrected_results_2792 = []

for model_name, predictions in model_predictions.items():
    corrected_results_2792.append({
        "Model": model_name,
        "MAE": mean_absolute_error(actual, predictions),
        "RMSE": np.sqrt(mean_squared_error(actual, predictions)),
        "MAPE_percent": (
            np.mean(np.abs((actual[nonzero_mask] - np.
 ↪asarray(predictions)[nonzero_mask]) / actual[nonzero_mask])) * 100
            if nonzero_mask.any() else np.nan
        ),
        "MAPE_observations": int(nonzero_mask.sum()),
        "Zero_actuals_excluded_from_MAPE": int((~nonzero_mask).sum())
    })

corrected_product_2792 = pd.DataFrame(corrected_results_2792)
display(corrected_product_2792.round(2))
```

```
           Model   MAE  RMSE  MAPE_percent  MAPE_observations  \
0  Moving Average  5.73  7.46         25.28                 90
1             SES  5.74  7.47         25.50                 90
2      Holt Trend  5.71  7.46         24.78                 90

   Zero_actuals_excluded_from_MAPE
0                                0
```

```
1                                   0
2                                   0
```

0.1.21 Corrected Evaluation Trial: Product 2792

After converting the training data to daily frequency and interpolating three missing training values, the forecasting results remained very similar to the original results. Holt's Linear Trend model achieved slightly lower MAE and MAPE values, although the differences between the models were small. This indicates that the missing training values had limited impact on this product's evaluation.

[67]:
```
product_id = 4425

train_df = common_period_data[product_id]["train"].sort_values("date")
test_df = common_period_data[product_id]["test"].sort_values("date")

train_daily_4425 = (
    train_df.set_index("date")[["sales"]]
    .asfreq("D")
)

train_daily_4425["sales"] = (
    train_daily_4425["sales"].interpolate(method="time")
)

print("Training calendar days:", len(train_daily_4425))
print("Missing training values after interpolation:",
      train_daily_4425["sales"].isna().sum())
print("Test records:", len(test_df))
print("Missing test sales:", test_df["sales"].isna().sum())
```

```
Training calendar days: 910
Missing training values after interpolation: 0
Test records: 90
Missing test sales: 0
```

0.1.22 Missing-Value Treatment: Product 4425

For product 4425, the training series was converted to daily frequency, and three missing training values were estimated using time-based linear interpolation. The 90 observed test records were retained without modification for model evaluation.

[68]:
```
train_values = train_daily_4425["sales"].to_numpy()
actual = test_df["sales"].to_numpy()
actual_dates = pd.DatetimeIndex(test_df["date"])
forecast_dates = pd.date_range(test_start, test_end, freq="D")
forecast_positions = forecast_dates.get_indexer(actual_dates)
if (forecast_positions < 0).any():
```

```
    raise ValueError("An observed test date falls outside the forecast␣
 ↪calendar")
nonzero_mask = actual != 0

model_predictions = {
    "Moving Average": np.full(
        len(forecast_dates), train_daily_4425["sales"].tail(7).mean()
    ),
    "SES": SimpleExpSmoothing(
        train_values, initialization_method="estimated"
    ).fit().forecast(len(forecast_dates)),
    "Holt Trend": Holt(
        train_values, initialization_method="estimated"
    ).fit(optimized=True).forecast(len(forecast_dates))
}

model_predictions = {
    model_name: np.asarray(predictions)[forecast_positions]
    for model_name, predictions in model_predictions.items()
}

corrected_results_4425 = []

for model_name, predictions in model_predictions.items():
    corrected_results_4425.append({
        "Model": model_name,
        "MAE": mean_absolute_error(actual, predictions),
        "RMSE": np.sqrt(mean_squared_error(actual, predictions)),
        "MAPE_percent": (
            np.mean(np.abs((actual[nonzero_mask] - np.
 ↪asarray(predictions)[nonzero_mask]) / actual[nonzero_mask])) * 100
            if nonzero_mask.any() else np.nan
        ),
        "MAPE_observations": int(nonzero_mask.sum()),
        "Zero_actuals_excluded_from_MAPE": int((~nonzero_mask).sum())
    })

corrected_product_4425 = pd.DataFrame(corrected_results_4425)
display(corrected_product_4425.round(2))
```

```
           Model    MAE   RMSE  MAPE_percent  MAPE_observations  \
0  Moving Average  43.76  56.28         16.55                 90
1             SES  42.93  55.09         17.21                 90
2      Holt Trend  43.02  55.30         16.87                 90

   Zero_actuals_excluded_from_MAPE
0                                0
```

```
1                                   0
2                                   0
```

0.1.23 Corrected Evaluation Trial: Product 4425

After converting the training data to daily frequency and interpolating three missing training values, the model performance remained broadly consistent with the original evaluation. Simple Exponential Smoothing achieved the lowest MAE and RMSE, while the Moving Average model achieved the lowest MAPE. The small differences suggest that the missing training values had limited impact on the results for this product.

[69]:
```
product_id = 2723

train_df = common_period_data[product_id]["train"].sort_values("date")
test_df = common_period_data[product_id]["test"].sort_values("date")

train_daily_2723 = (
    train_df.set_index("date")[["sales"]]
    .asfreq("D")
)

train_daily_2723["sales"] = (
    train_daily_2723["sales"].interpolate(method="time")
)

print("Training calendar days:", len(train_daily_2723))
print("Missing training values after interpolation:",
      train_daily_2723["sales"].isna().sum())
print("Test records:", len(test_df))
print("Missing test sales:", test_df["sales"].isna().sum())
```

```
Training calendar days: 1312
Missing training values after interpolation: 0
Test records: 90
Missing test sales: 0
```

0.1.24 Missing-Value Treatment: Product 2723

For product 2723, the training series was converted to daily frequency. The 30 missing training values were estimated using time-based linear interpolation. The original 90 test observations were retained without modification. Since this product has more missing training dates than the previously evaluated products, its model results will help assess the sensitivity of forecasting performance to the missing-value treatment.

[70]:
```
train_values = train_daily_2723["sales"].to_numpy()
actual = test_df["sales"].to_numpy()
actual_dates = pd.DatetimeIndex(test_df["date"])
forecast_dates = pd.date_range(test_start, test_end, freq="D")
forecast_positions = forecast_dates.get_indexer(actual_dates)
```

```
if (forecast_positions < 0).any():
    raise ValueError("An observed test date falls outside the forecast␣
 ↪calendar")
nonzero_mask = actual != 0

model_predictions = {
    "Moving Average": np.full(
        len(forecast_dates), train_daily_2723["sales"].tail(7).mean()
    ),
    "SES": SimpleExpSmoothing(
        train_values, initialization_method="estimated"
    ).fit().forecast(len(forecast_dates)),
    "Holt Trend": Holt(
        train_values, initialization_method="estimated"
    ).fit(optimized=True).forecast(len(forecast_dates))
}

model_predictions = {
    model_name: np.asarray(predictions)[forecast_positions]
    for model_name, predictions in model_predictions.items()
}

corrected_results_2723 = []

for model_name, predictions in model_predictions.items():
    corrected_results_2723.append({
        "Model": model_name,
        "MAE": mean_absolute_error(actual, predictions),
        "RMSE": np.sqrt(mean_squared_error(actual, predictions)),
        "MAPE_percent": (
            np.mean(np.abs((actual[nonzero_mask] - np.
 ↪asarray(predictions)[nonzero_mask]) / actual[nonzero_mask])) * 100
            if nonzero_mask.any() else np.nan
        ),
        "MAPE_observations": int(nonzero_mask.sum()),
        "Zero_actuals_excluded_from_MAPE": int((~nonzero_mask).sum())
    })

corrected_product_2723 = pd.DataFrame(corrected_results_2723)
display(corrected_product_2723.round(2))
```

```
            Model   MAE  RMSE  MAPE_percent  MAPE_observations  \
0  Moving Average  4.51  6.79         32.47                 90
1             SES  5.13  6.55         49.43                 90
2      Holt Trend  5.23  6.60         50.87                 90

   Zero_actuals_excluded_from_MAPE
```

```
0                                0
1                                0
2                                0
```

0.1.25 Corrected Evaluation Trial: Product 2723

After converting the training data to daily frequency and interpolating 30 missing training values, the Moving Average model achieved the lowest MAE and MAPE. Simple Exponential Smoothing and Holt's Linear Trend model produced higher percentage errors. The results indicate that model performance varies by product and that the handling of missing training dates may influence the evaluation. The comparison remains specific to this product.

[71]:
```
product_id = 4002

train_df = common_period_data[product_id]["train"].sort_values("date")
test_df = common_period_data[product_id]["test"].sort_values("date")

train_daily_4002 = (
    train_df.set_index("date")[["sales"]]
    .asfreq("D")
)

train_daily_4002["sales"] = (
    train_daily_4002["sales"].interpolate(method="time")
)

print("Training calendar days:", len(train_daily_4002))
print("Missing training values after interpolation:",
      train_daily_4002["sales"].isna().sum())
print("Test records:", len(test_df))
print("Missing test sales:", test_df["sales"].isna().sum())
```

```
Training calendar days: 559
Missing training values after interpolation: 0
Test records: 90
Missing test sales: 0
```

0.1.26 Missing-Value Treatment: Product 4002

For product 4002, the training data was converted to daily frequency, and 20 missing training values were estimated using time-based linear interpolation. The original 90 test observations were retained without modification. Forecasting performance will be evaluated using this prepared training series and the unchanged test data.

[72]:
```
train_values = train_daily_4002["sales"].to_numpy()
actual = test_df["sales"].to_numpy()
actual_dates = pd.DatetimeIndex(test_df["date"])
forecast_dates = pd.date_range(test_start, test_end, freq="D")
forecast_positions = forecast_dates.get_indexer(actual_dates)
```

```
if (forecast_positions < 0).any():
    raise ValueError("An observed test date falls outside the forecast␣
 ↪calendar")
nonzero_mask = actual != 0

model_predictions = {
    "Moving Average": np.full(
        len(forecast_dates), train_daily_4002["sales"].tail(7).mean()
    ),
    "SES": SimpleExpSmoothing(
        train_values, initialization_method="estimated"
    ).fit().forecast(len(forecast_dates)),
    "Holt Trend": Holt(
        train_values, initialization_method="estimated"
    ).fit(optimized=True).forecast(len(forecast_dates))
}

model_predictions = {
    model_name: np.asarray(predictions)[forecast_positions]
    for model_name, predictions in model_predictions.items()
}

corrected_results_4002 = []

for model_name, predictions in model_predictions.items():
    corrected_results_4002.append({
        "Model": model_name,
        "MAE": mean_absolute_error(actual, predictions),
        "RMSE": np.sqrt(mean_squared_error(actual, predictions)),
        "MAPE_percent": (
            np.mean(np.abs((actual[nonzero_mask] - np.
 ↪asarray(predictions)[nonzero_mask]) / actual[nonzero_mask])) * 100
            if nonzero_mask.any() else np.nan
        ),
        "MAPE_observations": int(nonzero_mask.sum()),
        "Zero_actuals_excluded_from_MAPE": int((~nonzero_mask).sum())
    })

corrected_product_4002 = pd.DataFrame(corrected_results_4002)
display(corrected_product_4002.round(2))
```

```
            Model    MAE   RMSE  MAPE_percent  MAPE_observations  \
0  Moving Average  33.98  43.09         60.26                 90
1             SES  33.37  42.96         57.87                 90
2      Holt Trend  34.21  43.43         60.53                 90

   Zero_actuals_excluded_from_MAPE
```

```
0                                0
1                                0
2                                0
```

0.1.27 Corrected Evaluation Trial: Product 4002

After converting the training data to daily frequency and interpolating 20 missing training values, Simple Exponential Smoothing achieved the lowest MAE, RMSE, and MAPE among the three evaluated models. However, the MAPE values remained relatively high, indicating substantial percentage errors for this product. These findings demonstrate that the best-performing forecasting method can vary across products.

[73]:
```
corrected_results_all = pd.concat([
    corrected_product_4067.assign(Product=4067),
    corrected_product_2792.assign(Product=2792),
    corrected_product_4425.assign(Product=4425),
    corrected_product_2723.assign(Product=2723),
    corrected_product_4002.assign(Product=4002)
], ignore_index=True)

corrected_summary = (
    corrected_results_all
    .groupby("Model")[["MAE", "RMSE", "MAPE_percent"]]
    .mean()
    .round(2)
    .reset_index()
)

display(corrected_summary)
```

```
            Model    MAE   RMSE  MAPE_percent
0      Holt Trend  20.85  26.78         36.20
1  Moving Average  21.02  27.16         32.35
2             SES  20.85  26.83         35.44
```

0.1.28 Corrected Overall Model Performance

The forecasting models were reevaluated using daily training series, with missing training values estimated through time-based linear interpolation. The original test observations were retained unchanged for five products with complete test-period coverage.

Holt's Linear Trend model achieved the lowest mean RMSE, while its mean MAE was tied with Simple Exponential Smoothing at the displayed precision. The Moving Average model achieved the lowest mean MAPE. The results indicate that model performance varies according to the evaluation metric, and no model was consistently superior across all three metrics.

[74]:
```
print("Columns:", complete_test_comparison.columns.tolist())
display(complete_test_comparison.head())
```

```
Columns: ['unique_id', 'MAE_MA', 'RMSE_MA', 'MAPE_percent_MA', 'MAE_SES',
'RMSE_SES', 'MAPE_percent_SES', 'MAE_Holt', 'RMSE_Holt', 'MAPE_Holt']
```

	unique_id	MAE_MA	RMSE_MA	MAPE_percent_MA	MAE_SES	RMSE_SES	\
0	4067	17.131286	22.171587	27.171960	17.067765	22.087203	
1	2792	5.729032	7.456723	25.278247	5.742092	7.470880	
2	4425	43.763238	56.284645	16.551979	42.936212	55.096643	
3	2723	4.508952	6.788491	32.469585	5.087782	6.532380	
4	4002	33.979905	43.089544	60.262730	33.297516	42.954045	

	MAPE_percent_SES	MAE_Holt	RMSE_Holt	MAPE_Holt
0	27.183817	16.081312	21.090445	27.967525
1	25.501790	5.728851	7.522875	24.327854
2	17.216723	43.870717	56.112967	16.887874
3	48.872192	5.189967	6.581048	50.302500
4	57.538512	34.783843	43.639320	62.481509

[75]:
```
original_summary = pd.DataFrame([
    {
        "Model": "Moving Average",
        "MAE": complete_test_comparison["MAE_MA"].mean(),
        "RMSE": complete_test_comparison["RMSE_MA"].mean(),
        "MAPE_percent": complete_test_comparison["MAPE_percent_MA"].mean()
    },
    {
        "Model": "SES",
        "MAE": complete_test_comparison["MAE_SES"].mean(),
        "RMSE": complete_test_comparison["RMSE_SES"].mean(),
        "MAPE_percent": complete_test_comparison["MAPE_percent_SES"].mean()
    },
    {
        "Model": "Holt Trend",
        "MAE": complete_test_comparison["MAE_Holt"].mean(),
        "RMSE": complete_test_comparison["RMSE_Holt"].mean(),
        "MAPE_percent": complete_test_comparison["MAPE_Holt"].mean()
    }
])

original_summary["Evaluation"] = "Original"

corrected_summary_copy = corrected_summary.copy()
corrected_summary_copy["Evaluation"] = "Corrected"

comparison_before_after = pd.concat(
    [original_summary, corrected_summary_copy],
    ignore_index=True
)
```

```
display(
    comparison_before_after[
        ["Evaluation", "Model", "MAE", "RMSE", "MAPE_percent"]
    ].round(2)
)
```

	Evaluation	Model	MAE	RMSE	MAPE_percent
0	Original	Moving Average	21.02	27.16	32.35
1	Original	SES	20.83	26.83	35.26
2	Original	Holt Trend	21.13	26.99	36.39
3	Corrected	Holt Trend	20.85	26.78	36.20
4	Corrected	Moving Average	21.02	27.16	32.35
5	Corrected	SES	20.85	26.83	35.44

0.1.29 Comparison Before and After Missing-Value Treatment

The original and corrected evaluations were compared across five products with complete test-period coverage. Missing training dates were handled by converting the training series to daily frequency and applying time-based linear interpolation, while the original test observations were retained.

The overall metrics changed only slightly. The Moving Average results remained unchanged at two decimal places. Simple Exponential Smoothing showed a small increase in MAE and MAPE, while Holt's Linear Trend model showed modest improvements in the three metrics. The overall ranking by evaluation metric remained broadly consistent: Moving Average achieved the lowest mean MAPE, and Holt's Linear Trend model achieved the lowest mean RMSE. These findings apply to the selected products and should not be generalized to all e-commerce products.

0.1.30 Forecasting Methodology: Fixed Moving-Average Baseline

The Moving Average model uses the average sales from the final seven observed training days to generate a constant forecast for the 90-day testing period. The forecast is not updated as new test-period observations become available. This approach serves as a simple baseline against which Simple Exponential Smoothing and Holt's Linear Trend models are compared.

Model performance is evaluated using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and Mean Absolute Percentage Error (MAPE).

0.1.31 Scope and Limitations of the Evaluation

This study evaluates three forecasting methods—Moving Average, Simple Exponential Smoothing, and Holt's Linear Trend—using five selected products from the Rohlik Sales Forecasting dataset. The products were selected based on data coverage, and the evaluation uses a common 90-day testing period from 5 March to 2 June 2024.

Missing values in the training series were estimated using time-based linear interpolation, while the observed test data was retained unchanged. The Moving Average method uses the final seven training days to produce a fixed forecast. The models are evaluated using MAE, RMSE, and MAPE. MAPE excludes zero actual sales; corrected product tables report the observation count used and zero actuals excluded. Product IDs occur in one warehouse only, so product forecasts

cannot compare the same product across warehouses. Warehouse totals are descriptive and reflect history length and assortment.

The findings are limited to the selected products and evaluation period. The dataset does not directly measure inventory costs, stockout reduction, or business profitability. Therefore, the results indicate forecasting performance rather than proven improvements in inventory outcomes.

[76]:
```
final_results = (
    corrected_summary
    .sort_values("Model")
    .reset_index(drop=True)
)

display(final_results.round(2))
```

	Model	MAE	RMSE	MAPE_percent
0	Holt Trend	20.85	26.78	36.20
1	Moving Average	21.02	27.16	32.35
2	SES	20.85	26.83	35.44

0.1.32 Key Findings from the Forecasting Analysis

The evaluation compared Moving Average, Simple Exponential Smoothing (SES), and Holt's Linear Trend using five selected products over a common 90-day testing period.

Holt's Linear Trend achieved the lowest mean RMSE of 26.78, while its mean MAE was 20.85. The Moving Average model achieved the lowest mean MAPE of 32.35%. SES recorded a mean MAE of 20.85 and mean RMSE of 26.83.

The results show that forecasting performance varies by evaluation metric and product. Although Holt's Linear Trend performed well on absolute error measures, Moving Average produced the lowest average percentage error. The differences in mean MAE and RMSE were small.

These findings suggest that simple forecasting methods can provide useful demand estimates for the selected products, but model choice should depend on the forecasting objective and the type of error that matters most. The results do not directly establish improvements in inventory costs, stock availability, or profitability.

[77]:
```
model_error_comparison = pd.DataFrame({
    "Product": complete_test_comparison["unique_id"],
    "MAE_Moving_Average": complete_test_comparison["MAE_MA"],
    "MAE_SES": complete_test_comparison["MAE_SES"],
    "MAE_Holt": complete_test_comparison["MAE_Holt"]
})

display(model_error_comparison.round(2))
```

	Product	MAE_Moving_Average	MAE_SES	MAE_Holt
0	4067	17.13	17.07	16.08
1	2792	5.73	5.74	5.73
2	4425	43.76	42.94	43.87

```
3      2723                4.51      5.09      5.19
4      4002               33.98     33.30     34.78
```

0.1.33 Product-Level Comparison of Forecasting Errors

The product-level MAE comparison shows that the best-performing forecasting method differs across the selected products. Holt's Linear Trend achieved the lowest MAE for product 4067, while Simple Exponential Smoothing performed best for products 4425 and 4002. The Moving Average model performed best for product 2723. The errors for product 2792 were very similar across all three methods.

These results indicate that forecasting performance depends on the product's sales behaviour. Therefore, the study does not identify a universally superior method. The findings are limited to the five evaluated products and require broader testing before being generalised.

[78]:
```python
from scipy.stats import friedmanchisquare

friedman_stat, friedman_p = friedmanchisquare(
    complete_test_comparison["MAE_MA"],
    complete_test_comparison["MAE_SES"],
    complete_test_comparison["MAE_Holt"]
)

print(f"Friedman test statistic: {friedman_stat:.4f}")
print(f"P-value: {friedman_p:.4f}")

if friedman_p < 0.05:
    print("Result: Statistically significant difference detected.")
else:
    print("Result: No statistically significant difference detected.")
```

```
Friedman test statistic: 0.4000
P-value: 0.8187
Result: No statistically significant difference detected.
```

0.1.34 Statistical Comparison of Forecasting Models: Friedman Test

A Friedman test was conducted to compare the MAE values of Moving Average, Simple Exponential Smoothing, and Holt's Linear Trend across five selected products. The test produced a statistic of 0.4000 and a p-value of 0.8187. At the 5% significance level, the result was not statistically significant, so the null hypothesis was not rejected.

The analysis did not identify sufficient statistical evidence that the forecasting methods differed in MAE across the selected products. However, the test was based on only five products, which limits its statistical power. Therefore, the result should not be interpreted as proof that the models perform equivalently.

[ ]:
```python

```

[ ]:
```python

```

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

[ ]:

0.2 Expanded Warehouse-Stratified Rolling-Origin Evaluation

0.2.1 Objective and evaluation criteria

This next-stage evaluation compares the 7-day Moving Average baseline, Simple Exponential Smoothing (SES), and Holt's linear trend across eligible product–warehouse series. Products are sampled within each warehouse and training-period coefficient-of-variation tertile, so the comparison covers different sales variability profiles. A product is eligible only when it has at least 365 observed training dates before 5 March 2024 and an observed record on every date in the shared 90-day evaluation window (5 March–2 June 2024).

Use three expanding-window folds, each forecasting the next 30 calendar days. Missing training dates are interpolated by time within each fold; missing actuals are not filled and the sample requires full test coverage. MAE and RMSE use all actuals. MAPE excludes zero actuals, with included/excluded counts reported. Results are macro-averaged across product series so high-volume products do not dominate; warehouse results remain descriptive because products are not shared between warehouses. This is a single 90-day backtest, not proof of future operational benefit.

[79]:
```
# Build a reproducible warehouse- and variability-stratified sample for the␣
↪shared test period.
import numpy as np
import pandas as pd

EVAL_START = pd.Timestamp("2024-03-05")
EVAL_END = pd.Timestamp("2024-06-02")
EVAL_DATES = pd.date_range(EVAL_START, EVAL_END, freq="D")
MIN_TRAIN_OBSERVATIONS = 365
MAX_PRODUCTS_PER_WAREHOUSE_VARIABILITY = 3
RANDOM_SEED = 42

# Group once rather than repeatedly scanning the multi-million-row source table.
```

```
eligibility_rows = []
for (warehouse_name, product_id), series in df_clean.groupby(["warehouse",␣
 ↪"unique_id"], sort=False):
    series = series.sort_values("date")
    train = series.loc[series["date"] < EVAL_START]
    test = series.loc[series["date"].between(EVAL_START, EVAL_END)]
    observed_test_dates = pd.DatetimeIndex(test["date"].drop_duplicates())
    full_test_coverage = len(observed_test_dates) == len(EVAL_DATES) and␣
 ↪observed_test_dates.equals(EVAL_DATES)
    if len(train) >= MIN_TRAIN_OBSERVATIONS and full_test_coverage:
        mean_sales = train["sales"].mean()
        std_sales = train["sales"].std()
        eligibility_rows.append({
            "warehouse": warehouse_name,
            "unique_id": product_id,
            "training_observations": len(train),
            "training_mean_sales": mean_sales,
            "training_cv": std_sales / mean_sales if mean_sales > 0 else np.nan,
            "test_observations": len(test)
        })

eligible_eval_products = pd.DataFrame(eligibility_rows)
if eligible_eval_products.empty:
    raise ValueError("No product series meet the training-history and␣
 ↪complete-test coverage rules.")

# Rank first to make tertile assignment deterministic even when CV values tie.
eligible_eval_products["variability_group"] = ""
for warehouse, idx in eligible_eval_products.groupby("warehouse").groups.
 ↪items():
    group = eligible_eval_products.loc[idx]
    if len(group) < 3:
        continue
    ranks = group["training_cv"].rank(method="first")
    eligible_eval_products.loc[idx, "variability_group"] = pd.qcut(
        ranks, q=3, labels=["Low variability", "Medium variability", "High␣
 ↪variability"]
    ).astype(str).to_numpy()

# Exclude warehouses with too few eligible candidates to support the three␣
 ↪variability strata.
stratifiable = eligible_eval_products.
 ↪loc[eligible_eval_products["variability_group"] != ""].copy()
selected_eval_products = pd.concat([
    group.sample(
        n=min(MAX_PRODUCTS_PER_WAREHOUSE_VARIABILITY, len(group)),
```

```
        random_state=RANDOM_SEED
    )
    for _, group in stratifiable.groupby(["warehouse", "variability_group"],␣
 ↪observed=True)
], ignore_index=True)
if selected_eval_products.empty:
    raise ValueError("No warehouse has enough eligible products for␣
 ↪variability-stratified sampling.")

selection_summary = (
    selected_eval_products.groupby(["warehouse", "variability_group"],␣
 ↪observed=True)
    .agg(products_selected=("unique_id", "count"),
         mean_training_observations=("training_observations", "mean"))
    .reset_index()
)
print("Eligible product-warehouse series:", len(eligible_eval_products))
print("Selected product-warehouse series:", len(selected_eval_products))
print("Warehouses represented:", selected_eval_products["warehouse"].nunique(),␣
 ↪"of", df_clean["warehouse"].nunique())
display(selection_summary)
display(selected_eval_products.sort_values(["warehouse", "variability_group",␣
 ↪"unique_id"]))

# Keep only the selected series for subsequent folds instead of repeatedly␣
 ↪scanning all records.
selected_series_data = df_clean.merge(
    selected_eval_products[["warehouse", "unique_id"]],
    on=["warehouse", "unique_id"],
    how="inner"
)
selected_series_map = {
    key: group.set_index("date")["sales"].sort_index()
    for key, group in selected_series_data.groupby(["warehouse", "unique_id"],␣
 ↪sort=False)
}
```

```
Eligible product-warehouse series: 1324
Selected product-warehouse series: 63
Warehouses represented: 7 of 7
```

	warehouse	variability_group	products_selected	\
0	Brno_1	High variability	3	
1	Brno_1	Low variability	3	
2	Brno_1	Medium variability	3	
3	Budapest_1	High variability	3	
4	Budapest_1	Low variability	3	
5	Budapest_1	Medium variability	3	

```
6   Frankfurt_1    High variability                3
7   Frankfurt_1     Low variability                3
8   Frankfurt_1  Medium variability                3
9      Munich_1    High variability                3
10     Munich_1     Low variability                3
11     Munich_1  Medium variability                3
12     Prague_1    High variability                3
13     Prague_1     Low variability                3
14     Prague_1  Medium variability                3
15     Prague_2    High variability                3
16     Prague_2     Low variability                3
17     Prague_2  Medium variability                3
18     Prague_3    High variability                3
19     Prague_3     Low variability                3
20     Prague_3  Medium variability                3

    mean_training_observations
0                    911.333333
1                   1192.666667
2                   1040.000000
3                    728.666667
4                   1173.000000
5                    968.000000
6                    660.333333
7                    581.000000
8                    615.666667
9                    466.333333
10                   867.000000
11                   479.000000
12                  1220.000000
13                  1112.666667
14                  1288.000000
15                  1179.333333
16                  1271.000000
17                  1276.333333
18                   753.333333
19                  1174.666667
20                  1051.000000

   warehouse  unique_id  training_observations  training_mean_sales  \
0     Brno_1        897                    406            17.176527
2     Brno_1       1755                   1020           142.988922
1     Brno_1       2896                   1308            35.295719
5     Brno_1       2365                   1287           100.904981
3     Brno_1       4644                   1142            82.640771
..       …          …                      …                 …
58  Prague_3        781                   1308            42.107469
57  Prague_3       1392                   1308           348.157064
```

```
62  Prague_3       1529                  1286           91.365187
61  Prague_3       2343                   610          120.765934
60  Prague_3       3462                  1257          118.101217

    training_cv  test_observations   variability_group
0      0.537666                90    High variability
2      0.597806                90    High variability
1      0.782046                90    High variability
5      0.280116                90     Low variability
3      0.328035                90     Low variability
..          …                 …                 …
58     0.348374                90     Low variability
57     0.373523                90     Low variability
62     0.480317                90  Medium variability
61     0.410080                90  Medium variability
60     0.437515                90  Medium variability

[63 rows x 7 columns]
```

[ ]:
```python
# Three expanding-window, 30-day folds; all model forecasts are aligned to calendar dates.
from sklearn.metrics import mean_absolute_error, mean_squared_error
from statsmodels.tsa.holtwinters import SimpleExpSmoothing, Holt

FOLD_DAYS = 30
fold_starts = [EVAL_START + pd.Timedelta(days=i * FOLD_DAYS) for i in range(3)]
model_names = ["Moving Average", "SES", "Holt Trend"]
fold_metric_rows = []

for item in selected_eval_products.itertuples(index=False):
    series = selected_series_map[(item.warehouse, item.unique_id)]

    for fold_number, fold_start in enumerate(fold_starts, start=1):
        fold_end = fold_start + pd.Timedelta(days=FOLD_DAYS - 1)
        observed_train = series.loc[series.index < fold_start]
        actual_series = series.reindex(pd.date_range(fold_start, fold_end, freq="D"))
        if actual_series.isna().any():
            raise ValueError(f"Missing actual test dates for product {item.unique_id}, fold {fold_number}.")

        train_daily = observed_train.asfreq("D")
        train_daily = train_daily.interpolate(method="time").ffill().bfill()
        train_values = train_daily.to_numpy(dtype=float)
        actual = actual_series.to_numpy(dtype=float)
        zero_mask = actual == 0
        predictions_by_model = {
```

```
            "Moving Average": np.full(FOLD_DAYS, train_daily.tail(7).mean()),
            "SES": SimpleExpSmoothing(
                train_values, initialization_method="estimated"
            ).fit().forecast(FOLD_DAYS),
            "Holt Trend": Holt(
                train_values, initialization_method="estimated"
            ).fit(optimized=True).forecast(FOLD_DAYS)
        }

        for model_name in model_names:
            predictions = np.asarray(predictions_by_model[model_name],␣
 ↪dtype=float)
            nonzero = ~zero_mask
            fold_metric_rows.append({
                "warehouse": item.warehouse,
                "unique_id": item.unique_id,
                "variability_group": item.variability_group,
                "fold": fold_number,
                "fold_start": fold_start,
                "fold_end": fold_end,
                "Model": model_name,
                "MAE": mean_absolute_error(actual, predictions),
                "RMSE": np.sqrt(mean_squared_error(actual, predictions)),
                "MAPE_percent": (
                    np.mean(np.abs((actual[nonzero] - predictions[nonzero]) /␣
 ↪actual[nonzero])) * 100
                    if nonzero.any() else np.nan
                ),
                "MAPE_observations": int(nonzero.sum()),
                "Zero_actuals_excluded_from_MAPE": int(zero_mask.sum())
            })

rolling_fold_results = pd.DataFrame(fold_metric_rows)
# Pool the equal-length folds within each series, then macro-average series.
def _aggregate_rolling_product_model(group):
    mape_counts = group["MAPE_observations"].to_numpy(dtype=float)
    mape_values = group["MAPE_percent"].to_numpy(dtype=float)
    valid = np.isfinite(mape_values) & (mape_counts > 0)
    weighted_mape = float(np.average(mape_values[valid],␣
 ↪weights=mape_counts[valid])) if valid.any() else np.nan
    return pd.Series({
        "MAE": group["MAE"].mean(),
        "RMSE": float(np.sqrt(np.mean(np.square(group["RMSE"].
 ↪to_numpy(dtype=float))))),
        "MAPE_percent": weighted_mape,
        "MAPE_observations": group["MAPE_observations"].sum(),
```

```
        "Zero_actuals_excluded_from_MAPE":␣
  ↪group["Zero_actuals_excluded_from_MAPE"].sum(),
        "folds_evaluated": group["fold"].nunique()
    })

product_model_scores = (
    rolling_fold_results.groupby(
        ["warehouse", "unique_id", "variability_group", "Model"], sort=False
    ).apply(_aggregate_rolling_product_model).reset_index()
)

warehouse_model_summary = (
    product_model_scores.groupby(["warehouse", "Model"], as_index=False)
    .agg(product_series=("unique_id", "nunique"),
         macro_MAE=("MAE", "mean"),
         macro_RMSE=("RMSE", "mean"),
         macro_MAPE_percent=("MAPE_percent", "mean"))
    .sort_values(["warehouse", "Model"])
)
overall_model_summary = (
    product_model_scores.groupby("Model", as_index=False)
    .agg(product_series=("unique_id", "nunique"),
         macro_MAE=("MAE", "mean"),
         macro_RMSE=("RMSE", "mean"),
         macro_MAPE_percent=("MAPE_percent", "mean"))
    .sort_values("macro_MAE")
)

print("Rolling-origin folds:", [(s.date(), (s + pd.Timedelta(days=29)).date())␣
  ↪for s in fold_starts])
print("Products evaluated:", selected_eval_products["unique_id"].nunique())
print("Per-product/model fold count:", rolling_fold_results.
  ↪groupby(["unique_id", "Model"])["fold"].nunique().value_counts().to_dict())
print()
print("Overall macro-average (each product series equally weighted):")
display(overall_model_summary.round(2))
print()
print("Warehouse-level macro-average:")
display(warehouse_model_summary.round(2))
print()
print("Per-product model scores:")
display(product_model_scores.sort_values(["warehouse", "unique_id", "Model"]).
  ↪round(2))
```

0.2.2 Interpreting this stage

Review the selected sample table first: confirm which warehouses and variability groups are represented, and note any underrepresented strata. Then compare overall and warehouse-level macro-averaged metrics, checking that each product has three folds. MAPE excludes zero-sales days; its counts are summed over folds. The score summaries are descriptive, and a model with the smallest average error is not automatically best for every warehouse or product. Treat this as an expanded, common-calendar backtest; it still does not model inventory availability, lead times, or stockout costs.

[ ]:

[ ]:

0.3 Paired Model Comparison and Naive Baseline

This section adds a last-observation-carried-forward naive forecast to the same three 30-day folds used above. The naive value is the final daily training value after the notebook's existing time interpolation. It then compares all four methods on the same product–warehouse series. MAE is the primary paired comparison; RMSE is secondary. Friedman tests check for an overall difference, followed by paired Wilcoxon tests with Holm correction for the MAE pairwise comparisons. These tests are evidence about this sampled backtest only, not proof that models are equivalent or future performance is guaranteed.

Run this cell after the expanded rolling-origin evaluation cell. It does not overwrite prior result tables.

[82]:
```
# Evaluate a last-observation-carried-forward naive forecast on the exact same␣
 ↪folds.
# Then add paired, product-level tests; existing evaluation rows and outputs␣
 ↪are preserved.
import itertools
import numpy as np
import pandas as pd
from scipy.stats import friedmanchisquare, wilcoxon
from statsmodels.stats.multitest import multipletests

_required = {"warehouse", "unique_id", "variability_group", "Model", "MAE",␣
 ↪"RMSE", "MAPE_percent"}
if "fold_metric_rows" not in globals() or not fold_metric_rows:
    raise RuntimeError("Run the expanded rolling-origin evaluation cell first;␣
 ↪fold_metric_rows is unavailable.")

existing_folds = pd.DataFrame(fold_metric_rows)
_missing = _required.difference(existing_folds.columns)
if _missing:
    raise ValueError(f"Evaluation fold results are missing expected columns:␣
 ↪{sorted(_missing)}")
```

```
naive_fold_rows = []
for item in selected_eval_products.itertuples(index=False):
    series = selected_series_map[(item.warehouse, item.unique_id)]
    for fold_number, fold_start in enumerate(fold_starts, start=1):
        fold_end = fold_start + pd.Timedelta(days=FOLD_DAYS - 1)
        observed_train = series.loc[series.index < fold_start]
        daily_train = observed_train.asfreq("D").interpolate(method="time").
 ↪ffill().bfill()
        actual_series = series.reindex(pd.date_range(fold_start, fold_end,␣
 ↪freq="D"))
        if actual_series.isna().any():
            raise ValueError(f"Missing actual test dates for product {item.
 ↪unique_id}, fold {fold_number}.")

        actual = actual_series.to_numpy(dtype=float)
        prediction = np.full(len(actual), float(daily_train.iloc[-1]))
        abs_error = np.abs(actual - prediction)
        squared_error = (actual - prediction) ** 2
        nonzero = actual != 0
        naive_fold_rows.append({
            "warehouse": item.warehouse,
            "unique_id": item.unique_id,
            "variability_group": item.variability_group,
            "Model": "Naive (last value)",
            "fold": fold_number,
            "MAE": float(abs_error.mean()),
            "RMSE": float(np.sqrt(squared_error.mean())),
            "MAPE_percent": float((abs_error[nonzero] / np.
 ↪abs(actual[nonzero])).mean() * 100) if nonzero.any() else np.nan,
            "MAPE_observations": int(nonzero.sum()),
            "Zero_actuals_excluded_from_MAPE": int((~nonzero).sum()),
        })

all_fold_results = pd.concat([existing_folds, pd.DataFrame(naive_fold_rows)],␣
 ↪ignore_index=True, sort=False)

# Aggregate the three equal-length folds per product/model. Combining squared␣
 ↪RMSEs
# yields the RMSE over the concatenated 90-day test period.
def _aggregate_product_model(group):
    mape_counts = group.get("MAPE_observations", pd.Series(np.ones(len(group)),␣
 ↪index=group.index)).fillna(0).to_numpy(dtype=float)
    mape_values = group["MAPE_percent"].to_numpy(dtype=float)
    valid = np.isfinite(mape_values) & (mape_counts > 0)
```

```
    weighted_mape = float(np.average(mape_values[valid],␣
 ↪weights=mape_counts[valid])) if valid.any() else np.nan
    return pd.Series({
        "MAE": group["MAE"].mean(),
        "RMSE": float(np.sqrt(np.mean(np.square(group["RMSE"].
 ↪to_numpy(dtype=float))))),
        "MAPE_percent": weighted_mape,
        "folds_evaluated": len(group),
    })

baseline_product_model_scores = (
    all_fold_results.groupby(["warehouse", "unique_id", "variability_group", ␣
 ↪"Model"], sort=False)
    .apply(_aggregate_product_model)
    .reset_index()
)

print("Naive baseline rows:", len(naive_fold_rows), "(expected", ␣
 ↪len(selected_eval_products) * len(fold_starts), ")")
_complete_model_counts = baseline_product_model_scores.groupby(["warehouse", ␣
 ↪"unique_id"])["Model"].nunique()
print("Product series with all four models:", int(_complete_model_counts.eq(4).
 ↪sum()), "of", len(_complete_model_counts))
display(baseline_product_model_scores.groupby("Model").agg(
    product_series=("unique_id", "nunique"),
    macro_MAE=("MAE", "mean"),
    macro_RMSE=("RMSE", "mean"),
    macro_MAPE_percent=("MAPE_percent", "mean"),
).sort_values("macro_MAE"))

# MAE is the prespecified primary paired outcome: each row is one product-
warehouse series.
mae_wide = baseline_product_model_scores.pivot(index=["warehouse", ␣
 ↪"unique_id"], columns="Model", values="MAE")
model_order = ["Moving Average", "SES", "Holt Trend", "Naive (last value)"]
mae_wide = mae_wide.reindex(columns=model_order).dropna()
if len(mae_wide) < 3:
    raise ValueError("Too few complete paired product series for the planned␣
 ↪tests.")

friedman_mae = friedmanchisquare(*(mae_wide[name].to_numpy() for name in␣
 ↪model_order))
print(f"\nFriedman test across four models (primary MAE): n={len(mae_wide)}, ␣
 ↪statistic={friedman_mae.statistic:.4f}, p={friedman_mae.pvalue:.6g}")

pairwise_rows = []
```

```
for model_a, model_b in itertools.combinations(model_order, 2):
    differences = mae_wide[model_a].to_numpy() - mae_wide[model_b].to_numpy()
    if np.allclose(differences, 0):
        statistic, p_value = 0.0, 1.0
    else:
        result = wilcoxon(mae_wide[model_a], mae_wide[model_b],␣
 ↪alternative="two-sided", method="auto")
        statistic, p_value = float(result.statistic), float(result.pvalue)
    pairwise_rows.append({
        "Model A": model_a,
        "Model B": model_b,
        "Median MAE difference (A-B)": float(np.median(differences)),
        "Wilcoxon statistic": statistic,
        "raw p-value": p_value,
    })
pairwise_results = pd.DataFrame(pairwise_rows)
pairwise_results["Holm-adjusted p-value"] = multipletests(pairwise_results["raw␣
 ↪p-value"], method="holm")[1]
display(pairwise_results)

rmse_wide = baseline_product_model_scores.pivot(index=["warehouse",␣
 ↪"unique_id"], columns="Model", values="RMSE").reindex(columns=model_order).
 ↪dropna()
friedman_rmse = friedmanchisquare(*(rmse_wide[name].to_numpy() for name in␣
 ↪model_order))
print(f"Secondary Friedman test across four models (RMSE): n={len(rmse_wide)},␣
 ↪statistic={friedman_rmse.statistic:.4f}, p={friedman_rmse.pvalue:.6g}")
print("Interpret lower MAE/RMSE as better. Treat these tests as sample-specific␣
 ↪evidence; do not infer equivalence from a non-significant p-value.")
```

```
Naive baseline rows: 189 (expected 189 )
Product series with all four models: 63 of 63
```

Model	product_series	macro_MAE	macro_RMSE	macro_MAPE_percent
SES	63	46.725234	63.426705	53.965865
Holt Trend	63	46.959098	63.709093	54.532856
Naive (last value)	63	48.279668	65.932492	62.345208
Moving Average	63	51.942665	70.700584	50.471826

```
Friedman test across four models (primary MAE): n=63, statistic=61.5143,
p=2.7904e-13
```

	Model A	Model B	Median MAE difference (A-B)	\
0	Moving Average	SES	0.562638	
1	Moving Average	Holt Trend	0.307658	
2	Moving Average	Naive (last value)	-2.917365	
3	SES	Holt Trend	-0.038600	

```
4             SES  Naive (last value)                  -2.006754
5      Holt Trend  Naive (last value)                  -1.993824

   Wilcoxon statistic  raw p-value  Holm-adjusted p-value
0               613.0     0.006847               0.020540
1               691.0     0.029991               0.029991
2               480.0     0.000301               0.001203
3               630.0     0.009658               0.020540
4               312.0     0.000002               0.000011
5               338.0     0.000004               0.000022

Secondary Friedman test across four models (RMSE): n=63, statistic=68.1429,
p=1.06624e-14
Interpret lower MAE/RMSE as better. Treat these tests as sample-specific
evidence; do not infer equivalence from a non-significant p-value.
```

[ ]:

0.4 Warehouse-Level Practical Comparison

This section summarizes the sampled product-series errors separately for each warehouse and estimates paired MAE differences against SES. Each product series receives equal weight. The bootstrap intervals resample product series within each warehouse; because only a small stratified sample was selected per warehouse, treat these intervals as descriptive uncertainty for this sample, not as proof of dataset-wide performance. A positive `MAE reduction vs SES` means the comparator has higher error than SES. No new significance claims are made here.

[83]:
```
# Requires the four-model product-level results from the paired comparison cell␣
↪above.
import numpy as np
import pandas as pd

if "baseline_product_model_scores" not in globals():
    raise RuntimeError("Run the paired model comparison cell first.")

_scores = baseline_product_model_scores.copy()
_required_columns = {"warehouse", "unique_id", "Model", "MAE", "RMSE"}
_missing_columns = _required_columns.difference(_scores.columns)
if _missing_columns:
    raise ValueError(f"Product-level scores are missing columns:␣
↪{sorted(_missing_columns)}")
if _scores.duplicated(["warehouse", "unique_id", "Model"]).any():
    raise ValueError("Expected one aggregated score per warehouse, product, and␣
↪model.")

_model_order = ["SES", "Holt Trend", "Naive (last value)", "Moving Average"]
_warehouse_summary = (
    _scores.groupby(["warehouse", "Model"], as_index=False)
```

```
    .agg(product_series=("unique_id", "nunique"), macro_MAE=("MAE", "mean"),␣
 ↪macro_RMSE=("RMSE", "mean"))
)
_warehouse_summary["Model"] = pd.Categorical(_warehouse_summary["Model"],␣
 ↪categories=_model_order, ordered=True)
_warehouse_summary = _warehouse_summary.sort_values(["warehouse", "Model"])
print("Warehouse-level macro-average errors (products equally weighted):")
display(_warehouse_summary.round(2))

_mae_wide = _scores.pivot(index=["warehouse", "unique_id"], columns="Model",␣
 ↪values="MAE")
if not set(_model_order).issubset(_mae_wide.columns):
    raise ValueError("Expected all four forecasting models in the results.")
_mae_wide = _mae_wide[_model_order].dropna()

# Descriptive paired differences: positive means SES has lower MAE than the␣
 ↪comparator.
_rng = np.random.default_rng(2025)
_BOOTSTRAPS = 10000
_comparators = ["Holt Trend", "Naive (last value)", "Moving Average"]
_paired_rows = []
for warehouse_name, group in _mae_wide.groupby(level="warehouse", sort=True):
    for comparator in _comparators:
        ses_errors = group["SES"].to_numpy(dtype=float)
        other_errors = group[comparator].to_numpy(dtype=float)
        differences = other_errors - ses_errors
        n = len(differences)
        bootstrap_means = np.empty(_BOOTSTRAPS, dtype=float)
        for b in range(_BOOTSTRAPS):
            bootstrap_means[b] = _rng.choice(differences, size=n, replace=True).
 ↪mean()
        comparator_mean = float(other_errors.mean())
        reduction = float(differences.mean())
        _paired_rows.append({
            "warehouse": warehouse_name,
            "comparison": f"{comparator} minus SES",
            "product_series": n,
            "mean_MAE_reduction_vs_SES": reduction,
            "bootstrap_95pct_CI_low": float(np.quantile(bootstrap_means, 0.
 ↪025)),
            "bootstrap_95pct_CI_high": float(np.quantile(bootstrap_means, 0.
 ↪975)),
            "products_SES_lower_MAE_pct": float(np.mean(differences > 0) * 100),
            "reduction_pct_of_comparator_MAE": float(reduction /␣
 ↪comparator_mean * 100) if comparator_mean else np.nan,
        })
```

```
_warehouse_paired = pd.DataFrame(_paired_rows)
print()
print("Paired MAE differences by warehouse (positive reduction favors SES):")
display(_warehouse_paired.round(2))

print()
print("Interpretation: use the warehouse table to describe where model errors␣
 ↪differ. The paired reduction and bootstrap interval describe the selected␣
 ↪products only; intervals overlapping zero indicate uncertainty about the␣
 ↪direction in this small sample, not model equivalence.")
```

```
Warehouse-level macro-average errors (products equally weighted):
```

	warehouse	Model	product_series	macro_MAE	macro_RMSE
3	Brno_1	SES	9	47.87	66.68
0	Brno_1	Holt Trend	9	47.50	66.53
2	Brno_1	Naive (last value)	9	51.61	73.37
1	Brno_1	Moving Average	9	49.34	66.88
7	Budapest_1	SES	9	87.86	108.35
4	Budapest_1	Holt Trend	9	87.83	108.32
6	Budapest_1	Naive (last value)	9	68.72	90.54
5	Budapest_1	Moving Average	9	107.46	132.99
11	Frankfurt_1	SES	9	10.93	14.14
8	Frankfurt_1	Holt Trend	9	11.06	14.22
10	Frankfurt_1	Naive (last value)	9	13.49	16.98
9	Frankfurt_1	Moving Average	9	11.00	14.17
15	Munich_1	SES	9	26.05	35.82
12	Munich_1	Holt Trend	9	26.97	36.91
14	Munich_1	Naive (last value)	9	30.33	40.93
13	Munich_1	Moving Average	9	27.88	38.30
19	Prague_1	SES	9	95.01	133.90
16	Prague_1	Holt Trend	9	95.35	134.30
18	Prague_1	Naive (last value)	9	112.24	154.27
17	Prague_1	Moving Average	9	85.55	121.02
23	Prague_2	SES	9	33.23	49.39
20	Prague_2	Holt Trend	9	33.81	49.94
22	Prague_2	Naive (last value)	9	28.26	41.36
21	Prague_2	Moving Average	9	55.76	85.44
27	Prague_3	SES	9	26.13	35.69
24	Prague_3	Holt Trend	9	26.20	35.73
26	Prague_3	Naive (last value)	9	33.30	44.07
25	Prague_3	Moving Average	9	26.62	36.10

```
Paired MAE differences by warehouse (positive reduction favors SES):
```

	warehouse	comparison	product_series	\
0	Brno_1	Holt Trend minus SES	9	

```
1        Brno_1  Naive (last value) minus SES              9
2        Brno_1      Moving Average minus SES              9
3     Budapest_1          Holt Trend minus SES              9
4     Budapest_1  Naive (last value) minus SES              9
5     Budapest_1      Moving Average minus SES              9
6    Frankfurt_1          Holt Trend minus SES              9
7    Frankfurt_1  Naive (last value) minus SES              9
8    Frankfurt_1      Moving Average minus SES              9
9       Munich_1          Holt Trend minus SES              9
10      Munich_1  Naive (last value) minus SES              9
11      Munich_1      Moving Average minus SES              9
12      Prague_1          Holt Trend minus SES              9
13      Prague_1  Naive (last value) minus SES              9
14      Prague_1      Moving Average minus SES              9
15      Prague_2          Holt Trend minus SES              9
16      Prague_2  Naive (last value) minus SES              9
17      Prague_2      Moving Average minus SES              9
18      Prague_3          Holt Trend minus SES              9
19      Prague_3  Naive (last value) minus SES              9
20      Prague_3      Moving Average minus SES              9

    mean_MAE_reduction_vs_SES  bootstrap_95pct_CI_low  \
0                       -0.37                   -0.91
1                        3.74                   -0.86
2                        1.47                   -3.51
3                       -0.03                   -0.55
4                      -19.15                  -59.85
5                       19.60                    0.92
6                        0.13                    0.00
7                        2.56                    1.37
8                        0.07                   -0.46
9                        0.91                    0.02
10                       4.28                    1.39
11                       1.83                    0.33
12                       0.34                   -0.01
13                      17.24                    4.27
14                      -9.45                  -35.20
15                       0.58                    0.04
16                      -4.97                  -21.32
17                      22.53                   -0.86
18                       0.07                   -0.16
19                       7.17                    1.53
20                       0.49                   -0.93

    bootstrap_95pct_CI_high  products_SES_lower_MAE_pct  \
0                     -0.05                       22.22
1                      8.45                       88.89
2                      8.08                       66.67
```

3	0.35	66.67
4	2.61	55.56
5	54.64	77.78
6	0.29	77.78
7	4.02	100.00
8	0.45	66.67
9	2.47	88.89
10	7.36	77.78
11	3.13	88.89
12	0.93	66.67
13	35.99	88.89
14	5.90	66.67
15	1.54	88.89
16	4.01	88.89
17	56.18	55.56
18	0.31	66.67
19	13.50	100.00
20	2.14	55.56

	reduction_pct_of_comparator_MAE
0	-0.78
1	7.25
2	2.97
3	-0.04
4	-27.86
5	18.24
6	1.21
7	18.98
8	0.63
9	3.39
10	14.12
11	6.55
12	0.36
13	15.36
14	-11.05
15	1.73
16	-17.59
17	40.40
18	0.25
19	21.54
20	1.83

Interpretation: use the warehouse table to describe where model errors differ. The paired reduction and bootstrap interval describe the selected products only; intervals overlapping zero indicate uncertainty about the direction in this small sample, not model equivalence.

[ ]: